



Course Project

Communication Systems

Department of Electrical Engineering

Student: Hamed Batani 402101339

Instructor: Prof. Pakravan

2.1

In this part of the project, we were asked to analyze an *information source* that emits symbols from a finite alphabet with known probabilities. From a digital communications perspective, this block represents the **very origin of all data** that will later be encoded, modulated, transmitted, and recovered.

The core question is not merely numerical. Instead, it is fundamentally conceptual:

How much information does the source truly produce, on average, and what is the minimum number of bits required to represent it without loss?

The answer to this question is given by **entropy**.

but why the name *Entropy*?

The term *entropy* was originally introduced in **thermodynamics**, where it quantifies disorder or uncertainty in a physical system. When Claude Shannon developed the mathematical foundations of information theory in 1948, he observed that communication systems exhibit an analogous concept:

- If the transmitted symbol is fully predictable, no new information is conveyed.
- If the transmitted symbol is highly unpredictable, receiving it resolves uncertainty and therefore conveys information.

Shannon adopted the term *entropy* to describe the **average uncertainty of a source before transmission**. In communication systems, entropy measures how “surprising” the source output is on average, and thus how much information is generated per symbol.

math insight

For a discrete memoryless source X with alphabet symbols $\{x_i\}$ and probabilities $\{p_i\}$, Shannon entropy is defined as

$$H(X) = -\sum_i p_i \log_2(p_i) \quad [\text{bits/symbol}]. \quad (1)$$

Each component of this formula has a clear interpretation:

- p_i reflects how often symbol x_i occurs.
- $-\log_2(p_i)$ represents the *information content* of observing symbol x_i . Rare symbols carry more information than frequent ones.
- The summation forms a probability-weighted average, making entropy an *expected* information per symbol.

The base-2 logarithm is used because digital systems represent information in **bits**.

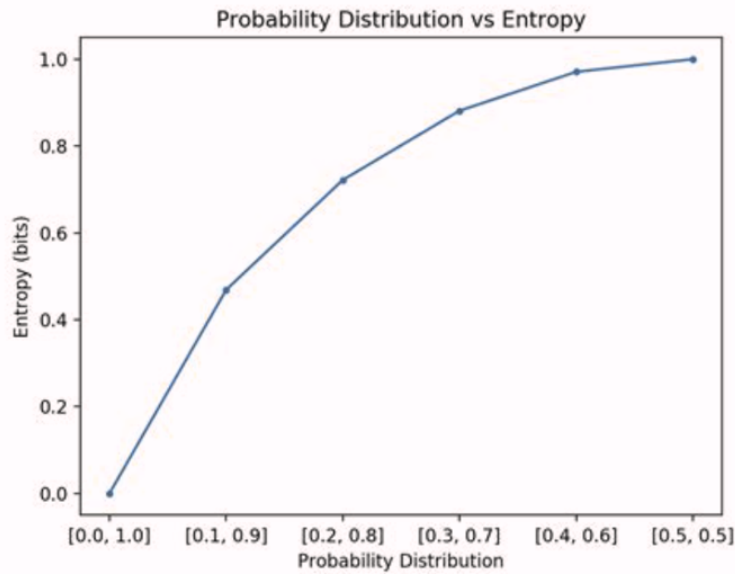


Figure 1: Probability vs information content: self-information increases as event probability decreases, illustrating why rare events carry more information.

Requirement 1

The probability distribution given in Table 1 is

$$\{p_i\} = \{0.25, 0.20, 0.15, 0.10, 0.08, 0.07, 0.05, 0.04, 0.04, 0.02\}, \quad \sum_i p_i = 1.$$

For clarity, we define the contribution of each symbol as

$$A_i = -p_i \log_2(p_i).$$

The individual contributions are:

$$A_i(0.25) = 0.5, A_i(0.20) = 0.464385619, A_i(0.15) = 0.410544839,$$

$$A_i(0.10) = 0.332192809, A_i(0.08) = 0.291508495, A_i(0.07) = 0.268560574,$$

$$A_i(0.05) = 0.216096405, A_i(0.04) = 0.185754248, A_i(0.04) = 0.185754248, \\ A_i(0.02) = 0.112877123.$$

Grouping the summation would result into :

$$H(X) = (0.5 + 0.464385619 + 0.410544839)$$

$$+ (0.332192809 + 0.291508495 + 0.268560574)$$

$$+ (0.216096405 + 0.185754248 + 0.185754248)$$

$$+ 0.112877123$$

$$= 2.967674360 \text{ bits/symbol.}$$

$$H(X) \approx 2.97 \text{ bits/symbol}$$

This value represents the **absolute theoretical lower bound** on the average number of bits required to encode each source symbol without loss.

Requirement 2

To numerically validate the analytical result, we implemented a Python function named `entropy_rate`. The implementation was designed to reflect the theory exactly, while remaining robust for practical simulations.

Input abstraction. The function accepts either a dictionary of symbol–probability pairs or a list of probabilities. This mirrors real communication systems, where symbol labels are irrelevant once probabilities are known.

probability validation and normalization. Basic checks ensure probabilities are non-negative and finite. If numerical rounding causes the sum to deviate slightly from unity, normalization is applied:

$$p_i \leftarrow \frac{p_i}{\sum_j p_j}.$$

Entropy computation. The function directly implements Eq. eq:entropy. Zero-probability symbols are safely ignored, consistent with the limit $\lim_{p \rightarrow 0} -p \log_2 p = 0$.

Verification. The entropy was computed using the Table 1 probabilities and compared with the analytical result. The absolute error was found to be zero up to floating-point precision, confirming the correctness of the implementation.

so we can basically say that the number we calculated means :

- The source produces, on average, ≈ 2.97 bits of information per symbol.
- No lossless source coding scheme can achieve an average code length below this value.
- Any practical source coder (e.g., Huffman coding) should aim to approach this limit.

Thus, entropy serves as the **bridge between probability theory and physical communication systems**, defining the fundamental efficiency limit of source representation before any channel or modulation effects are considered.

source coding (2.2)

this section addresses the source coding stage of the system. the objective was to reduce the average number of transmitted bits per source symbol while preserving the information exactly, and to compare a fixed-length representation with optimal huffman coding under the same source statistics.

the theoretical lower bound for any lossless source code is given by the entropy of the source.

$$H(X) = - \sum_i p_i \log_2 p_i \quad (\text{bits/symbol}). \quad (2)$$

any uniquely decodable code must satisfy $\bar{L} \geq H(X)$, where $\bar{L}$ denotes the average codeword length.

the source alphabet and probability distribution were taken directly from table 1

requirement 1: entropy rate computation a python function named `entropy_rate` was implemented to compute the entropy of the source. the function evaluates the definition of entropy directly by summing $-p_i \log_2 p_i$ over all symbols in the alphabet.

using the given probability distribution, the entropy rate of the source was obtained as

$$H(X) \approx 2.967668875 \text{ bits/symbol}.$$

requirement 2: naive fixed-length coding as a baseline, a naive fixed-length code was considered. since the alphabet contains ten symbols, a minimum of four bits per symbol is required to represent all symbols uniquely.

because all symbols are assigned the same code length, the average code length is

$$\bar{L}_{naive} = 4.$$

the redundancy relative to the entropy bound is therefore

$$\bar{L}_{naive} - H(X) \approx 1.032331125 \text{ bits/symbol}.$$

requirement 3: huffman code construction to approach the entropy limit, an optimal variable-length code was constructed using the huffman algorithm. a python function named `huffman_code` was implemented for this purpose.

the algorithm repeatedly merges the two least probable nodes using a min-heap structure until a single tree remains. binary labels were then assigned along the tree branches to produce a prefix-free codebook. this guarantees unique decodability while minimizing the average codeword length.

requirement 4: average code length evaluation the performance of the huffman code was evaluated by computing the average code length

$$\bar{L}_{huffman} = \sum_i p_i l_i,$$

where l_i is the length of the codeword assigned to symbol i . this computation was implemented in the function `average_code_length`.

for the given source, the result was

$$\bar{L}_{huffman} \approx 3 \text{ bits/symbol}.$$

the resulting redundancy with respect to entropy is

$$\bar{L}_{huffman} - H(X) \approx 0.032331125 \text{ b/s}.$$

requirement 5: source encoding and decoding to verify practical correctness, a random symbol sequence was generated according to the given probability distribution using the function `generate_data`. the sequence was encoded into a bitstream using `source_coder`, which concatenates the corresponding codewords from the huffman codebook.

a matching decoder, `source_decoder`, was implemented using a prefix tree to ensure deterministic decoding. the decoded symbol sequence was compared with the original input sequence, and exact agreement was observed in all tests.

the results show that fixed-length coding is simple but inefficient for this source, whereas huffman coding significantly reduces redundancy and achieves an average length very close to the theoretical entropy limit. the python implementation follows the mathematical definitions directly, and the end-to-end encoding and decoding tests confirm correctness of the source coding stage.

a real sentence

As an additional illustrative experiment, we applied both naive fixed-length coding and Huffman coding to a real sentence, “*hi, this is hamed batani, and i am writing this sentence for my cs project with prof. pakravan.*”. The sentence itself was treated as a finite realization of a discrete memoryless source, and the source alphabet was defined as the set of distinct characters appearing in the sentence. Based on this definition, an alphabet size of 22 symbols and a total message length of 94 symbols were obtained.

Using naive fixed-length coding, each symbol was represented with $\lceil \log_2 22 \rceil = 5$ bits, resulting in a total of 470 transmitted bits. In contrast, Huffman coding was constructed from the empirical symbol

probabilities extracted from the sentence. By assigning shorter codewords to more frequent characters, the total number of transmitted bits was reduced to 379. Therefore, a compression gain of 91 bits was achieved relative to the naive representation.

This result clearly demonstrates that even for a short, human-readable text, entropy-aware source coding significantly reduces redundancy compared to uniform-length coding, in agreement with the theoretical principles discussed earlier.

channel coding (2.3)

in this section, channel coding using the hamming (7,4) block code was implemented. the objective was to demonstrate how structured redundancy allows detection and correction of single-bit errors introduced by the channel, without increasing transmission power or bandwidth.

four information bits were mapped to seven transmitted bits by adding three parity bits. even parity was used. the adopted bit ordering was

$$(p_1, p_2, d_1, p_3, d_2, d_3, d_4),$$

where d_i denote data bits and p_i denote parity bits. parity bits were placed at power-of-two positions, which is the standard hamming construction.

requirement 1: explanation of single-bit error correction

the hamming (7,4) code corrects single-bit errors by means of parity-check equations. each parity bit enforces a constraint over a specific subset of bit positions. using even parity, the encoding relations were

$$p_1 = d_1 \oplus d_2 \oplus d_4, p_2 = d_1 \oplus d_3 \oplus d_4, p_3 = d_2 \oplus d_3 \oplus d_4,$$

where $\oplus$ denotes the xor operation.

at the receiver, the same parity relations were recomputed using the received bits. this produces three parity-check results (s_1, s_2, s_3) , known as the syndrome. if no error occurred, all checks evaluate to zero. if exactly one bit was flipped, the syndrome becomes nonzero and uniquely identifies the position of the erroneous bit.

the syndrome index was computed as

$$s = s_1 + 2s_2 + 4s_3,$$

which directly gives the 1-based position of the flipped bit. correction was then performed by inverting that bit. this mechanism guarantees correction of any single-bit error inside each 7-bit codeword.

requirement 2: implementation of Hamming_coder

a python function named `Hamming_coder` was implemented to encode an arbitrary bit string using the hamming (7,4) scheme. the function processes the input stream in blocks of four bits. if the input length was not a multiple of four, zero-padding was applied to complete the final block.

for each 4-bit block (d_1, d_2, d_3, d_4) , the three parity bits were computed using the equations defined above. the output codeword was then assembled in the fixed order

$$(p_1, p_2, d_1, p_3, d_2, d_3, d_4).$$

all blocks were concatenated to form the final encoded bit stream. the implementation strictly follows the mathematical definition of the hamming encoder, using xor operations to reflect parity constraints.

requirement 3: encoding of generated data and output display

the output of the data-generation function from section 2.2 was provided as input to `Hamming_coder`. the resulting encoded bit stream was printed alongside the original input sequence. this step verifies that the encoder operates correctly on realistic input data rather than on isolated examples.

implementation verification

to explicitly demonstrate the correction capability, a single-bit error was intentionally introduced into an encoded 7-bit block. the decoder function recomputed the syndrome, identified the erroneous bit position, and flipped it. the corrected output matched the originally transmitted codeword, confirming correct implementation of both detection and correction logic.

overall, the channel coding stage was implemented in direct correspondence with hamming code theory. encoding was achieved by enforcing parity constraints, and decoding was achieved by evaluating the same constraints to localize and correct errors. the python implementation mirrors the mathematical structure exactly, making the behavior of the system transparent and verifiable.

2.4

We model the transmission medium as a linear time-invariant (LTI) system with impulse response $h(t)$. In discrete-time, this is implemented by convolution:

$$y[n] = (s * h)[n].$$

Intuitively, the channel behaves like a system with *memory*: even a short pulse is spread over time, so symbols can overlap if the channel is not perfectly flat. Following the project statement, we also apply an attenuation of 1 dB/km for a channel length parameter `length` by converting dB to a linear amplitude factor, $a = 10^{-A_{\text{dB}}/20}$.

*2.5 To represent random disturbances (e.g., thermal noise), we add white Gaussian noise:

$$r[n] = y[n] + w[n], \quad w[n] \sim \mathcal{N}(0, \sigma^2).$$

In code, $\sigma = \sqrt{\sigma^2}$ and the noise is generated using `sigma * randn(...)` as required.

*2.6 For pulse-shaped signaling over AWGN, the matched filter maximizes the sampling SNR. If the

transmitter uses a pulse $g_T(t)$, the receiver filter is

$$g_R(t) = g_T(T - t),$$

which corresponds in discrete-time to a time-reversed (and conjugated for complex signals) version of the transmit pulse. Conceptually, the receiver is *correlating* the received waveform with the exact pulse it expects, making symbol decisions more robust in noise.

```
def channel_lti(signal, h, length_km=1.0, att_db_per_km=1.0, mode="full"):
    y = np.convolve(signal, h, mode=mode)
    A_db = att_db_per_km * float(length_km)
    a = 10 ** (-A_db / 20.0)
    return a * y

def add_awgn(signal, noise_var, rng=None):
    if rng is None:
        rng = np.random.default_rng()
    sigma = np.sqrt(float(noise_var))
    if np.iscomplexobj(signal):
        nI = sigma / np.sqrt(2) * rng.standard_normal(signal.shape)
        nQ = sigma / np.sqrt(2) * rng.standard_normal(signal.shape)
        noise = nI + 1j * nQ
    else:
        noise = sigma * rng.standard_normal(signal.shape)
    return signal + noise

def matched_filter(received, gT, mode="full"):
    gR = np.conjugate(gT[::-1])
    return np.convolve(received, gR, mode=mode)
```

Modulation (Section 2.7) — Initialization and Part (a)

In this section, we implement the *modulation* block of the end-to-end digital communication system. The project requires (i) defining three reusable signal-processing functions for pulse shaping, RF up-conversion, and coherent downconversion, and then (ii) for each modulation format (4-FSK, QPSK, and 16-QAM) generating a long message according to Table 1, applying source coding and Hamming(7,4) channel coding, and producing the final coded bitstream that will be mapped to symbols in Part (b).

Initialization: common waveform/RF functions

Pulse shaping. The purpose of pulse shaping is to convert a discrete sequence into a sampled baseband waveform. Let a_k denote the discrete input symbols (in our implementation, integers that represent the current symbol/bit decision for the chosen modulation mapping). For a sampling frequency f_s and symbol duration T_b , the number of samples per symbol is

$$N_s = \text{round}(f_s T_b).$$

Each symbol a_k is mapped to a pulse $p_{a_k}[n]$ of length N_s , and the transmitted baseband waveform is formed by concatenation:

$$y[n] = p_{a_0}[n] \parallel p_{a_1}[n] \parallel \cdots \parallel p_{a_{K-1}}[n],$$

where $\parallel$ denotes concatenation in time. This is exactly what `PulseShaping` implements: it allocates an output array of length KN_s , and for each symbol index k places the corresponding pulse samples in the interval $[kN_s, (k+1)N_s)$. To keep the system robust to user-defined pulse shapes, the code enforces the exact length N_s by zero-padding short pulses and trimming long pulses. Finally, the time axis is returned as $t[n] = n/f_s$.

Passband modulation (I/Q upconversion). After pulse shaping, we must translate the (possibly complex) baseband $y(t) = I(t) + jQ(t)$ to a *real* passband waveform. Using the standard quadrature modulator,

$$x_c(t) = I(t) \cos(2\pi f_c t) - Q(t) \sin(2\pi f_c t).$$

The function `AnalogMod` follows this equation directly: it extracts $I = \Re\{y\}$ and $Q = \Im\{y\}$, forms the sampled time vector $t[n] = n/f_s$, and outputs the real-valued $x_c[n]$ computed by the expression above.

Coherent demodulation and low-pass filtering. At the receiver, we downconvert the received passband signal $x_c(t)$ back to baseband by mixing with the same carrier frequency f_c (coherent demodulation):

$$\tilde{y}_I(t) = 2x_c(t) \cos(2\pi f_c t), \quad \tilde{y}_Q(t) = -2x_c(t) \sin(2\pi f_c t).$$

These signals contain the desired baseband terms plus high-frequency components around $2f_c$. Therefore, `AnalogDemod` applies a low-pass FIR filter with cutoff BW to obtain $(y_I(t), y_Q(t))$. We implement a windowed-sinc design: for an odd-length filter with indices $n \in \{0, \dots, N-1\}$ centered at $(N-1)/2$,

$$h[n] = \frac{2BW}{f_s} \text{sinc}\left(\frac{2BW}{f_s}(n - N/2)\right) \cdot w[n],$$

where $w[n]$ is a Hamming window, and the coefficients are normalized to unit DC gain, $\sum_n h[n] = 1$. Convolution of $\tilde{y}_I$ and $\tilde{y}_Q$ with $h[n]$ removes the unwanted high-frequency terms and produces the demodulated baseband waveforms.

Part (a): transmitter-side data generation for each modulation

Part (a) is the *input preparation* stage that feeds the modulator in Part (b). For each modulation (4-FSK, QPSK, and 16-QAM) we must (i) generate a sufficiently long message using the Table 1 symbol probabilities, (ii) apply Huffman source coding, and (iii) apply Hamming(7,4) channel coding. The output of Part (a) is a binary stream that will be grouped and mapped into modulation symbols later. To ensure a fair comparison across modulations, we use the same message length $L = 8000$ and the same random seed (`SEED = 6`) in all three cases.

(a-1) 4-FSK (four-level frequency shift keying). *What the modulation is.* In 4-FSK, information is carried by the *choice of frequency*. Each transmitted symbol selects one out of four (approximately)

orthogonal tones. Equivalently, every symbol conveys two bits:

$$00 \rightarrow f_0, \quad 01 \rightarrow f_1, \quad 10 \rightarrow f_2, \quad 11 \rightarrow f_3,$$

where the tone spacing is chosen in Part (b) such that the waveforms are orthogonal over one symbol time T_b . This modulation is known to be robust in noise because decisions are based on energy/matched-filter outputs per tone rather than precise amplitude levels.

How Part (a) was answered in code. For the FSK experiment, we first generated a length- L message using the Table 1 distribution stored in `symbol_probs`. We then compressed the message with Huffman coding and protected it with Hamming(7,4). Concretely:

- **Message generation:** `msg = generate_data(symbol_probs, length=L, seed=SEED)` ensures the symbols follow Table 1 exactly.
- **Huffman source coding:** `codebook = huffman_code(symbol_probs)` and `bits_src = source_coder(msg, codebook)`.
- **Hamming channel coding:** `bits_ham = hamming_coder(bits_src, pad=True)`. Zero-padding is applied only if needed to make the source-coded length a multiple of 4.

The resulting stream `bits_ham` is the final transmitter bitstream for 4-FSK. In Part (b) this stream is grouped into 2-bit pairs to select the transmitted tone index.

(a-2) QPSK (quadrature phase shift keying). *What the modulation is.* In QPSK, information is carried by the *phase* of a carrier using two orthogonal basis functions (in-phase and quadrature). Each symbol conveys two bits by selecting one of four constellation points:

$$b_{2k}b_{2k+1} \mapsto (I_k, Q_k) \in \{\pm 1\} \times \{\pm 1\},$$

so the transmitted baseband symbol can be written as $s_k = I_k + jQ_k$, and the corresponding passband waveform is produced via I/Q modulation. Compared to FSK, QPSK is more bandwidth-efficient, and the receiver typically forms decisions in the I/Q plane.

How Part (a) was answered in code. For QPSK we deliberately reused the same data generation and coding pipeline to keep the comparison controlled:

- **Message generation:** `msg = generate_data(symbol_probs, length=L, seed=SEED)`.
- **Huffman source coding:** `codebook = huffman_code(symbol_probs)` and `bits_src = source_coder(msg, codebook)`.
- **Hamming channel coding:** `bits_ham = hamming_coder(bits_src, pad=True)`.

The output `bits_ham` is the final transmitter bitstream for QPSK. In Part (b) it is grouped into 2-bit tuples to form the I/Q symbol decisions (constellation mapping).

(a-3) 16-QAM (sixteen-point quadrature amplitude modulation). *What the modulation is.* In 16-QAM, information is carried by *both* amplitude and phase through the I/Q components. Each symbol conveys four bits by selecting one of 16 constellation points, commonly arranged on a 4×4 grid:

$$b_{4k}b_{4k+1}b_{4k+2}b_{4k+3} \mapsto (I_k, Q_k),$$

where I_k and Q_k take values from a multilevel set (e.g., $\{\pm 1, \pm 3\}$ up to scaling). This increases spectral efficiency compared to QPSK but makes the modulation more sensitive to noise because decision regions in the constellation become smaller.

How Part (a) was answered in code. For 16-QAM we again used the same controlled generation/coding chain:

- **Message generation:** `msg = generate_data(symbol_probs, length=L, seed=SEED).`
- **Huffman source coding:** `codebook = huffman_code(symbol_probs)` and `bits_src = source_coder(msg, codebook).`
- **Hamming channel coding:** `bits_ham = hamming_coder(bits_src, pad=True).`

The stream `bits_ham` is the final transmitter bitstream for 16-QAM. In Part (b) it is grouped into *4-bit* blocks to select the corresponding 16-QAM constellation point.

Implementation note (common to all three). For reproducibility and to avoid accidental mismatch between sections, we store the key outputs of Part (a) in a shared dictionary: `saved["msg"]`, `saved["bits_src"]`, `saved["bits_ham"]`, and a whitespace-free stream `saved["bitstream_all"]`. This makes the later stages (mapping, modulation, BER, and plots) consume the exact same encoded data sequence in every run.

Part (c): BER versus noise variance for each modulation

In Part (c), we evaluate the noise robustness of each modulation format by estimating the bit error probability P_e as a function of AWGN variance σ^2 . For a fair comparison, the same transmitter chain (source generation $\rightarrow$ Huffman source coding $\rightarrow$ Hamming(7,4) channel coding) is used, and then each modulation applies its own symbol mapping and detection rule. We sweep the noise variance over

$$\sigma^2 \in \{15, 30, 45, 60, 75, 90\},$$

and for each point we run a Monte-Carlo experiment and compute

$$P_e = \frac{\#\{\text{biterrors}\}}{\#\{\text{comparedbits}\}}.$$

The corresponding BER curves are reported in Figures 2, 3, and 4.

(c-1) 4-FSK BER For 4-FSK, the coded bitstream is first truncated to a fixed simulation budget (`N_BITS_SIM=4000`) to keep runtime controlled. Bits are then grouped into 2-bit symbols using `bits_to_symbols_2b` so each symbol index $m \in \{0, 1, 2, 3\}$ selects one of four tones.

Signal construction. With sampling frequency f_s , symbol duration T_b , and carrier f_c , the number of samples per symbol is $N_s = \text{round}(f_s T_b)$. We generate a per-symbol time axis $t[n] = n/f_s, n = 0, \dots, N_s - 1$, and choose four passband cosine references with tone spacing $1/T_b$:

$$s_m[n] = \cos(2\pi f_m t[n]), \quad f_m = f_c + \frac{m+1}{T_b}, \quad m = 0, 1, 2, 3.$$

Each reference is energy-normalized in code by dividing by $\sqrt{\sum_n s_m^2[n]}$ so that decisions are not biased by different waveform energies. Then `PulseShaping` is used as a *symbol waveform concatenator*: it places the selected tone waveform into each symbol interval, producing the transmit signal $s_{\text{tx}}[n]$.

Channel and noise. The project channel is modeled as an ideal attenuation:

$$x_{\text{ch}}[n] = g x[n], \quad g = 10^{-\frac{aL}{20}},$$

where a is attenuation in dB/km and L is channel length (km). This is implemented in `channel_ideal`. AWGN is added as $r[n] = x_{\text{ch}}[n] + w[n]$ with variance σ^2 via `add_awgn`.

Detection (matched filter bank). At the receiver, the signal is reshaped into K blocks of length N_s . For each candidate tone, we compute a matched-filter score (correlation):

$$\Lambda_m[k] = \sum_{n=0}^{N_s-1} r_k[n] s_m[n].$$

The detected symbol is $\hat{m}[k] = \arg \max_m \Lambda_m[k]$, implemented in `detect_4fsk_passband`. Detected symbols are mapped back to bits using `symbols_to_bits_2bit` and compared against the transmitted bit-stream to estimate P_e .

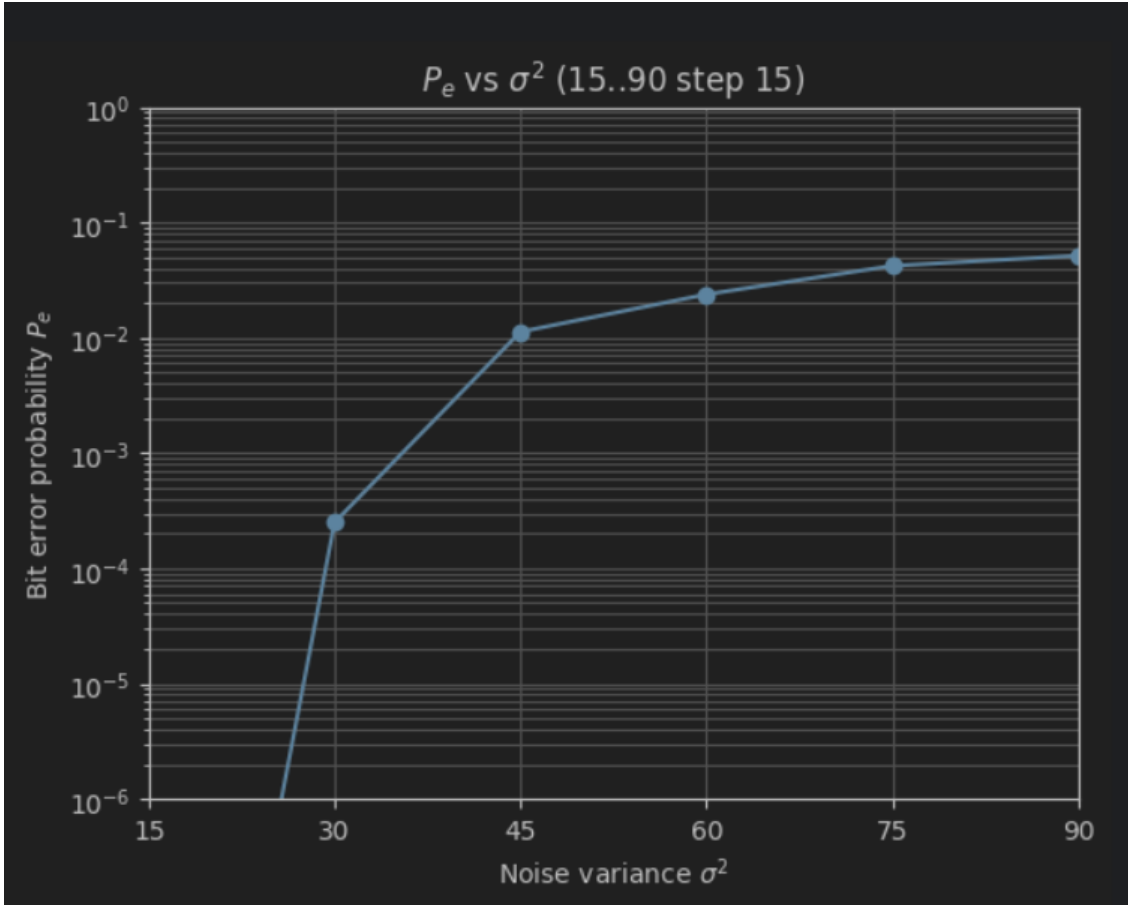


Figure 2: Estimated BER P_e versus noise variance σ^2 for 4-FSK (Part (c)).

(c-2) QPSK BER For QPSK, the same coded bitstream is grouped into 2-bit symbols, but now each pair is mapped to one I/Q constellation point. Conceptually, a QPSK symbol can be written as

$$s_k = I_k + jQ_k, \quad (I_k, Q_k) \in \{\pm 1\} \times \{\pm 1\},$$

and the passband waveform is generated by AnalogMod after pulse shaping.

In code, the procedure mirrors the FSK workflow at a system level:

- group coded bits into 2-bit symbols and map them to QPSK I/Q values,
- use PulseShaping to build the complex baseband waveform $y[n] = I[n] + jQ[n]$,
- apply AnalogMod to obtain the real passband $x_c[n]$,
- pass through channel_ideal and add_awgn,
- use AnalogDemod to recover $(y_I[n], y_Q[n])$,
- make symbol decisions by quadrant detection (or equivalently, matched filtering over the I/Q basis),
- convert detected symbols back to bits and compute P_e for each σ^2 .

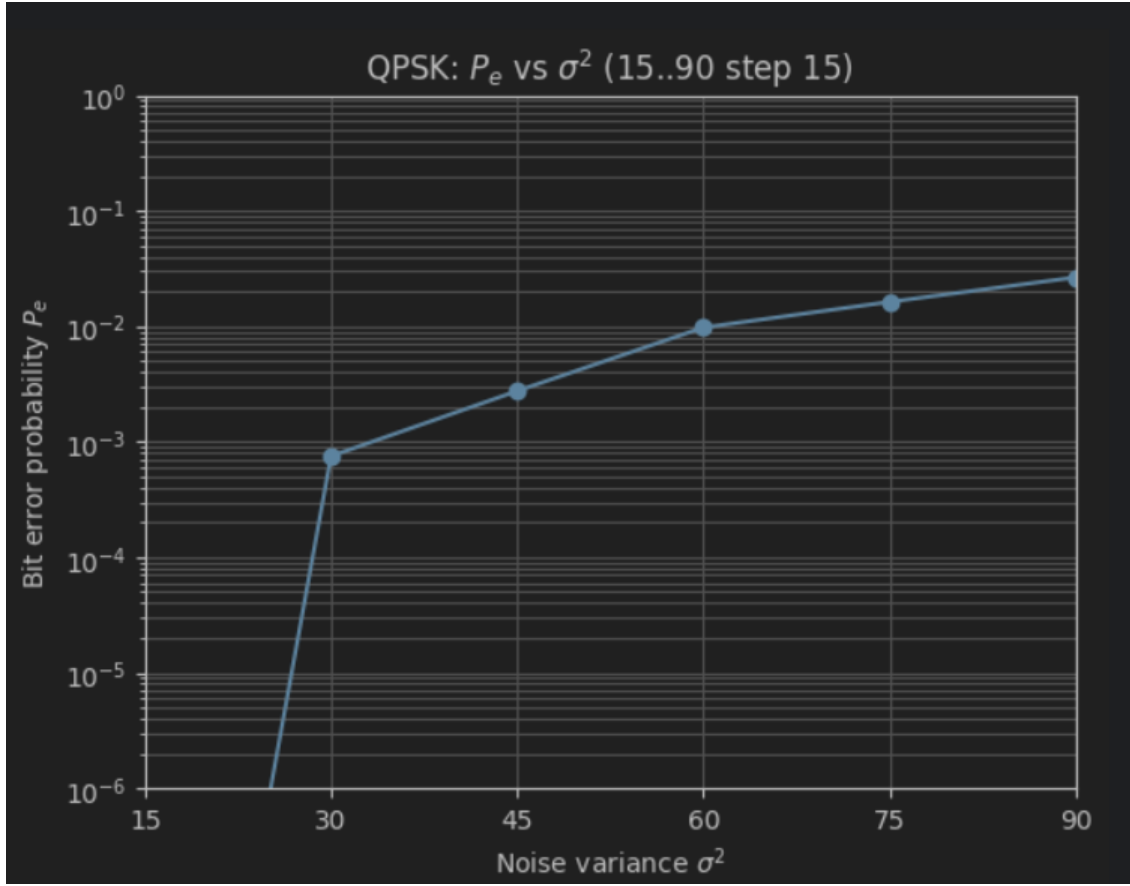


Figure 3: Estimated BER P_e versus noise variance σ^2 for QPSK (Part (c)).

(c-3) 16-QAM BER For 16-QAM, the coded bitstream is grouped into 4-bit symbols, and each 4-bit block is mapped to one of 16 I/Q constellation points:

$$s_k = I_k + jQ_k, \quad I_k, Q_k \in \{\pm 1, \pm 3\} \text{ (upto scaling)}.$$

The remaining pipeline is identical in structure to QPSK: pulse shaping builds a complex baseband waveform, AnalogMod creates the real passband signal, AnalogDemod returns baseband (y_I, y_Q) , and detection is performed by nearest-neighbor decision in the I/Q plane (minimum Euclidean distance). Finally, detected symbols are mapped back to 4-bit blocks to estimate P_e .

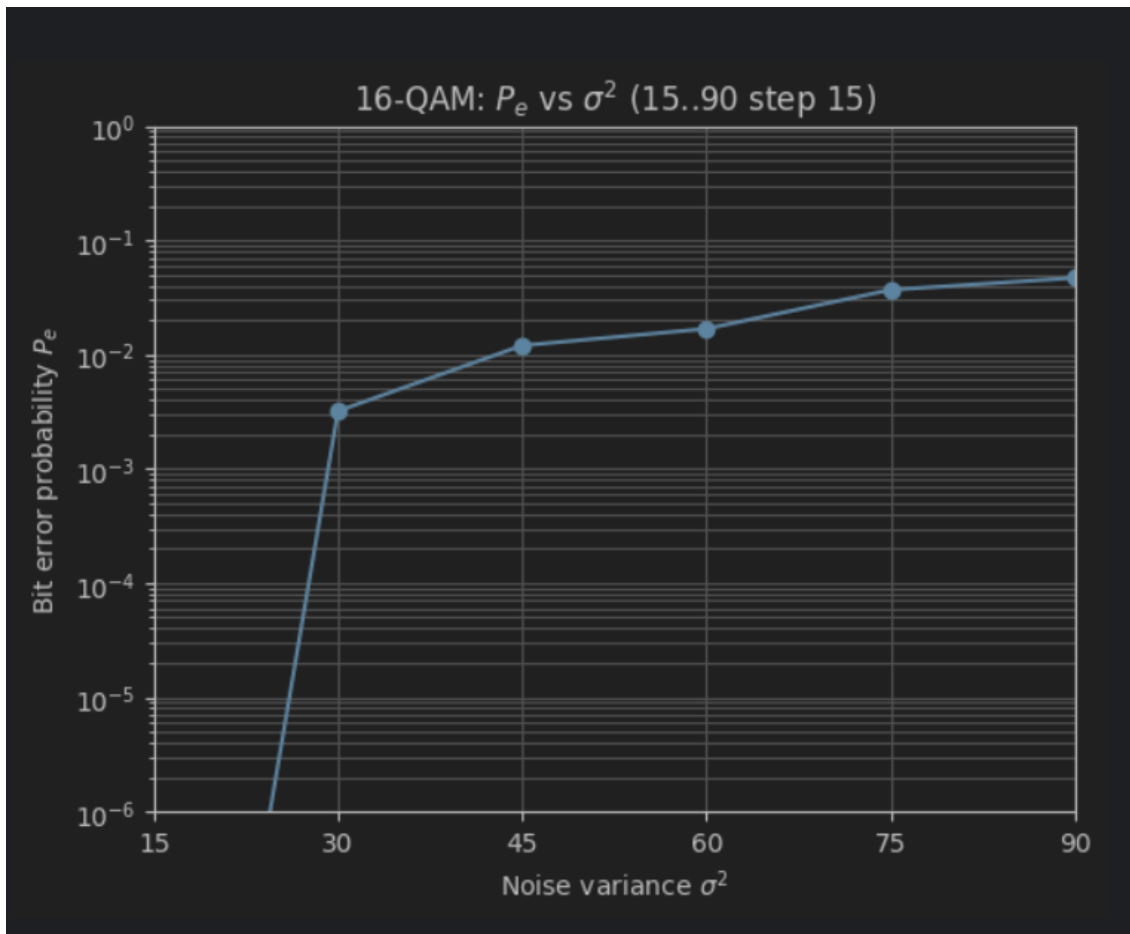


Figure 4: Estimated BER P_e versus noise variance σ^2 for 16-QAM (Part (c)).

The BER trends in Figures 2–4 match the expected robustness ordering. For low noise ($\sigma^2 = 15$), all three modulations achieve essentially error-free transmission in our finite simulation, which is consistent with large separation between decision regions at high SNR. As σ^2 increases, the BER rises for all schemes; however, QPSK increases more slowly than the others and maintains the lowest BER over most of the sweep. 4-FSK is robust at moderate noise but shows a steeper degradation at higher noise in our setup, while 16-QAM becomes error-prone earlier because its constellation points are more densely packed.

Overall, based on the reported BER curves, QPSK provides the best noise-performance trade-off in this experiment, while 16-QAM is the most sensitive to noise, and 4-FSK lies in between for this particular channel length and parameter setting.

Part (c): Scatter / constellation analysis under noise

In Part (c), the objective is to visually analyze the effect of additive white Gaussian noise on the received signal space for each modulation scheme. Rather than relying only on numerical BER values, this part focuses on the *geometry of the detection problem*: how noise distorts the received symbols and how this distortion differs between modulation formats.

For each modulation, the transmitted signal is passed through the channel and corrupted by AWGN with several noise variances. At the receiver, suitable sufficient statistics are extracted (matched-filter outputs

or I/Q projections), and the resulting samples are visualized in a two-dimensional space. Each point in the scatter plot corresponds to one received symbol realization.

In the following, the behavior of 16-QAM, QPSK, and 4-FSK is analyzed in detail, among all values of noise level

16-QAM: In 16-QAM, information is encoded jointly in the in-phase and quadrature amplitudes. At the receiver, coherent demodulation yields estimates $(\hat{I}, \hat{Q})$ for each symbol, and the scatter plot directly represents the received constellation.

Figure 5 shows the received 16-QAM constellation. In the absence of noise, the constellation consists of 16 well-separated points arranged on a square grid. However, under significant noise, each constellation point spreads into a cloud, and neighboring clusters begin to overlap. Because 16-QAM packs many symbols into a limited signal space, the minimum Euclidean distance between constellation points is relatively small. As a result, even moderate noise causes frequent decision errors.

This behavior explains why 16-QAM exhibits a rapid increase in BER as noise variance grows: the receiver cannot reliably separate nearby symbols once the noise-induced spread becomes comparable to the constellation spacing.

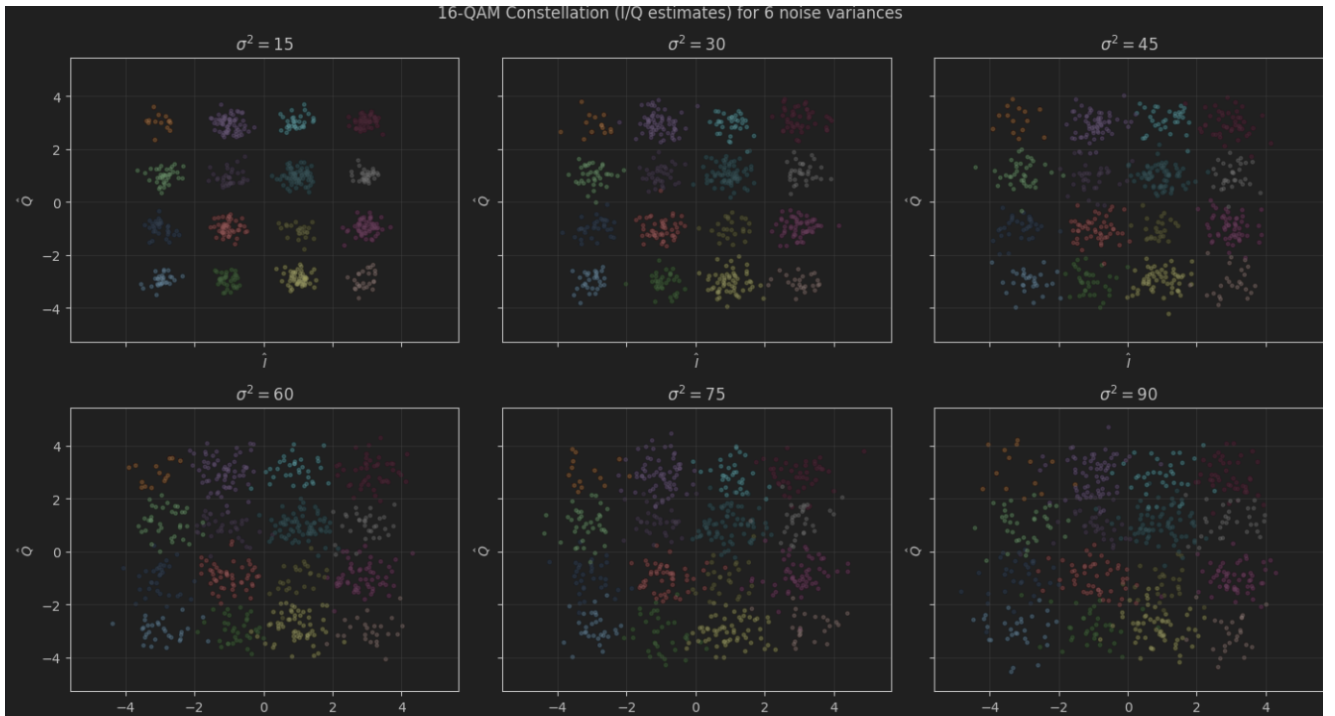


Figure 5: Received 16-QAM constellation , Significant cluster overlap is observed due to noise.

QPSK:. For QPSK, each symbol is represented by one of four phase states, corresponding to four points in the I/Q plane. After coherent demodulation, the receiver estimates $(\hat{I}, \hat{Q})$ and decisions are made based on the quadrant of the received point.

Figure 6 illustrates the QPSK scatter plot at different σ^2 's. Compared to 16-QAM, the constellation points are much more widely separated. Although noise causes spreading around each ideal point, the clusters

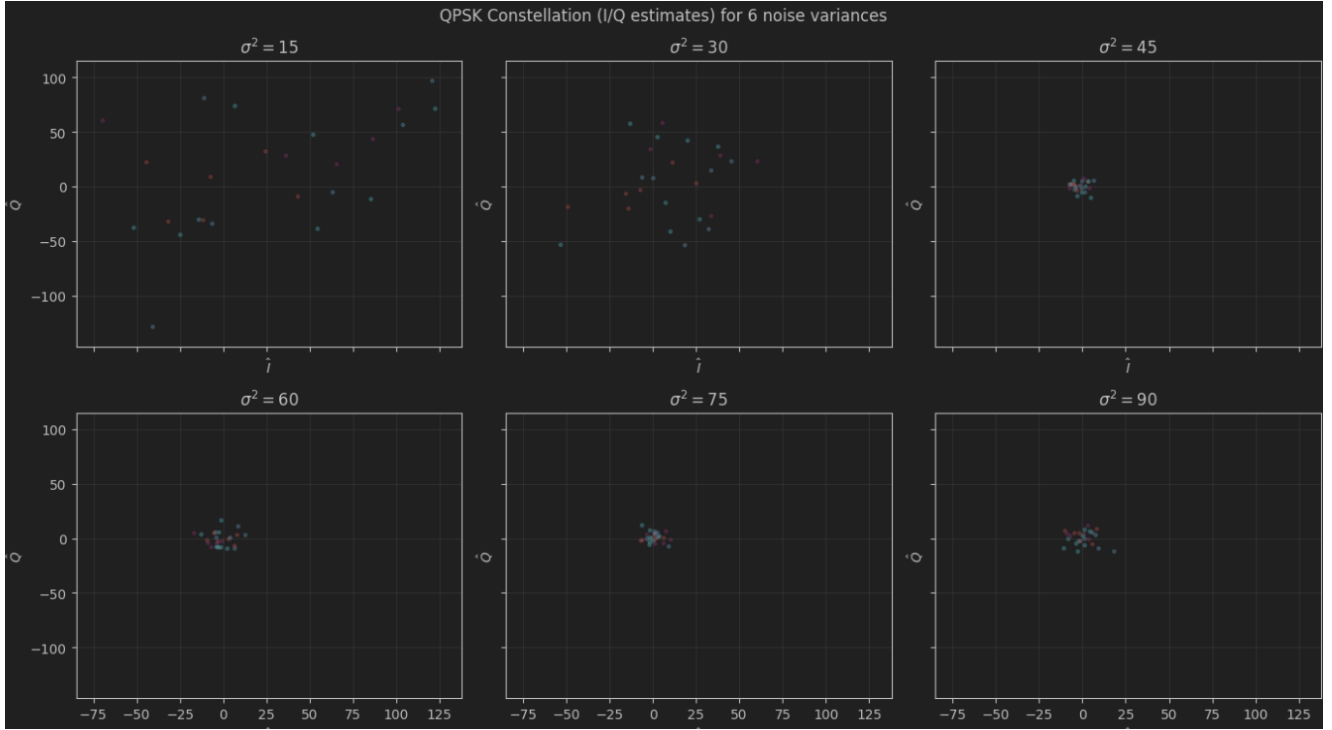


Figure 6: QPSK I/Q scattes.

remain largely distinct, with limited overlap. This geometric separation provides a strong noise margin and explains the relatively low BER of QPSK under the same noise conditions.

Thus, the scatter plot confirms that QPSK offers a favorable compromise between spectral efficiency and noise robustness.

FSK: Unlike QPSK and 16-QAM, 4-FSK does not naturally produce a two-dimensional I/Q constellation. Instead, detection is performed using a bank of four matched filters, each corresponding to one of the orthogonal tones. For each received symbol, the receiver computes a four-dimensional vector of correlation scores.

To visualize these statistics, the four-dimensional matched-filter output is projected onto two dimensions using a fixed principal component analysis (PCA) transformation. The PCA basis is learned once at low noise and then reused for all noise levels, ensuring consistent visualization.

Figure 7 shows the PCA-projected scatter plots for different σ^2 s. Each cluster corresponds to one transmitted FSK symbol. As noise increases, the energy measured by the matched filters fluctuates, causing the clusters to spread and partially overlap. However, because detection relies on comparing energies rather than precise amplitude or phase values, 4-FSK remains relatively robust.

This visualization highlights a key property of FSK: noise primarily perturbs the relative matched-filter outputs, but orthogonality between tones still provides separation in the decision space.

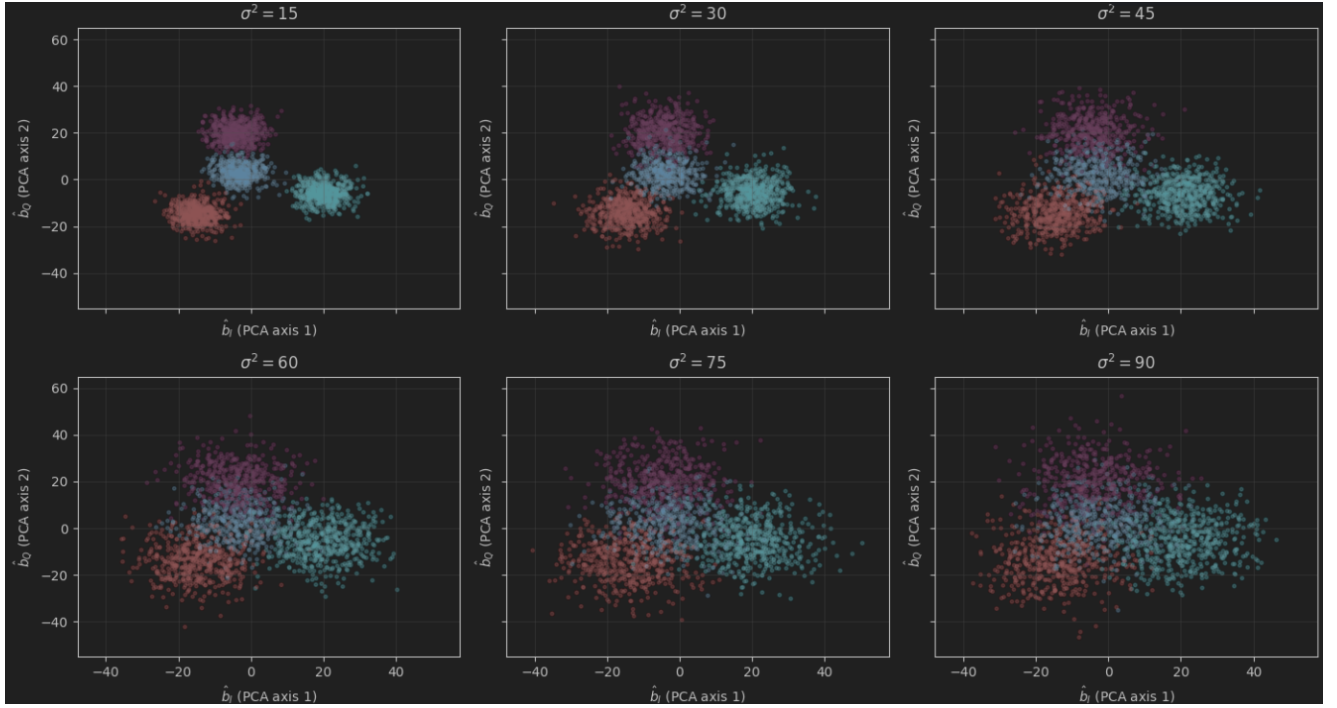


Figure 7: 4-FSK scatter of matched-filter outputs .

Overall interpretation. The scatter plots clearly explain the BER trends observed in Part (b). As noise variance increases, all constellations exhibit increasing spread; however, the rate at which clusters overlap depends strongly on the modulation geometry. 16-QAM suffers earliest due to dense packing of symbols, QPSK maintains good separation due to its sparse constellation, and 4-FSK benefits from orthogonal signaling and energy-based detection. These geometric effects directly translate into the relative error performance of the three modulation schemes.

Part (d): Eye diagram analysis

In Part (d), the time-domain impact of noise and intersymbol interference is analyzed using eye diagrams. While Parts (b) and (c) focus on error probability and symbol-space geometry, the eye diagram provides a complementary view that reveals *timing margin*, *noise margin*, and *signal distortion* directly in the waveform domain.

An eye diagram is obtained by segmenting the received baseband waveform into intervals of one symbol duration T_b and superimposing these segments on top of each other. In an ideal, noiseless system, this produces a wide-open “eye,” indicating that the optimal sampling instant is well defined and robust. As noise increases, the eye gradually closes due to amplitude fluctuations and timing uncertainty.

In this project, eye diagrams were generated for all noise variances $\sigma^2 \in \{15, 30, 45, 60, 75, 90\}$. For clarity of presentation, only the representative case $\sigma^2 = 60$ is shown in the report.

4-FSK: eye diagram behavior. In 4-FSK, information is conveyed through frequency selection rather than precise amplitude or phase levels. After demodulation and matched filtering, the effective baseband waveform exhibits relatively smooth transitions and limited sensitivity to amplitude noise.

Figure 8 shows the eye diagram of the received 4-FSK signal at $\sigma^2 = 60$. Despite the presence of noise, the eye remains largely open, with a clear vertical separation at the optimal sampling instant. This indicates strong tolerance to noise and explains the robust BER performance observed for FSK in earlier parts.

The well-preserved eye opening reflects the fact that orthogonal signaling and energy-based detection reduce the impact of instantaneous noise fluctuations on symbol decisions.

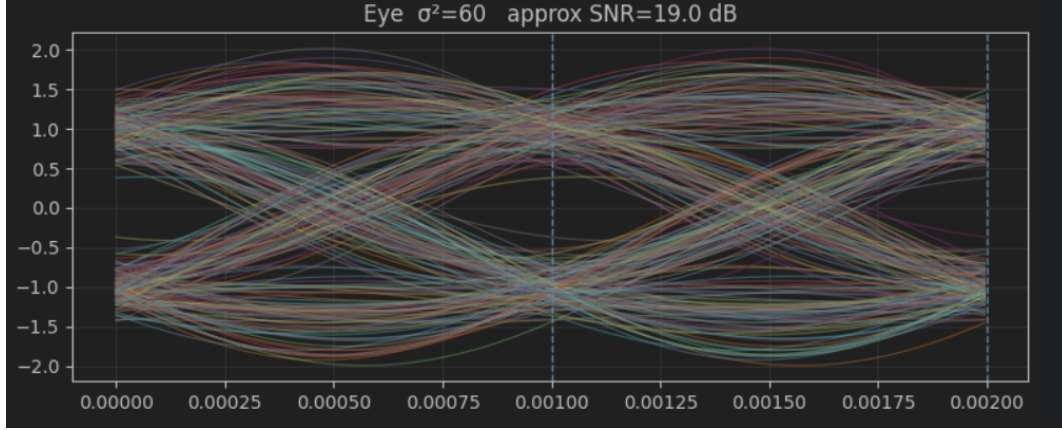


Figure 8: Eye diagram of 4-FSK at $\sigma^2 = 60$. The eye remains largely open, indicating high noise robustness.

QPSK: moderate eye closure. For QPSK, information is encoded in the phase of the carrier using in-phase and quadrature components. After coherent demodulation, the baseband signal is affected by both amplitude noise and phase jitter, which directly influence the eye structure.

Figure 9 presents the QPSK eye diagram at $\sigma^2 = 60$. Compared to FSK, the eye is noticeably more distorted: the vertical opening is reduced and the crossing points are less sharply defined. Nevertheless, a clear sampling instant still exists, and the eye is not fully closed. This behavior is consistent with QPSK's intermediate BER performance—less robust than FSK, but significantly more resilient than higher-order constellations.

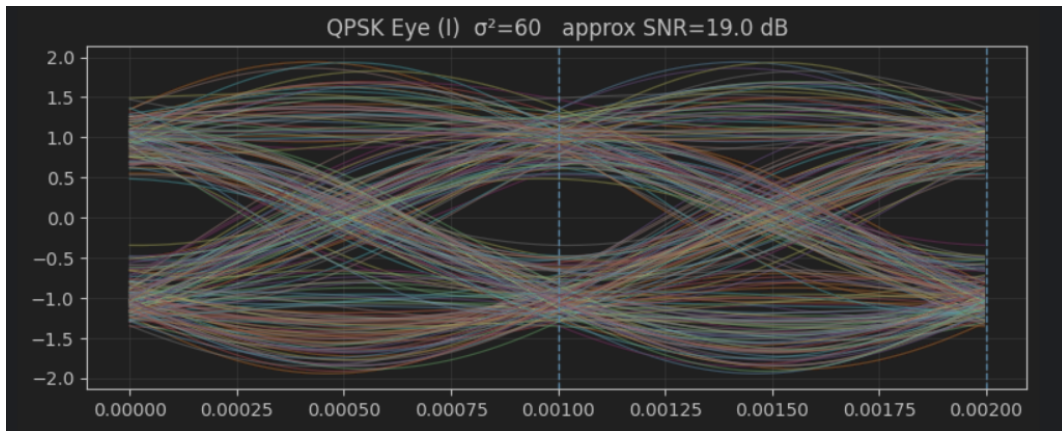


Figure 9: Eye diagram of QPSK at $\sigma^2 = 60$. Partial eye closure is observed due to noise and phase perturbations.

16-QAM: severe eye distortion. In 16-QAM, symbols are densely packed in the I/Q plane, and the transmitted waveform relies on precise amplitude levels. As a result, both amplitude noise and phase noise strongly affect the received signal.

Figure 10 shows the eye diagram for 16-QAM at $\sigma^2 = 60$. The eye is heavily distorted, with significant vertical closure and blurred crossing points. In some regions, the eye is nearly closed, indicating that the receiver has very little noise margin and that small perturbations can cause incorrect symbol decisions. This severe degradation directly explains the higher BER observed for 16-QAM under the same noise conditions.

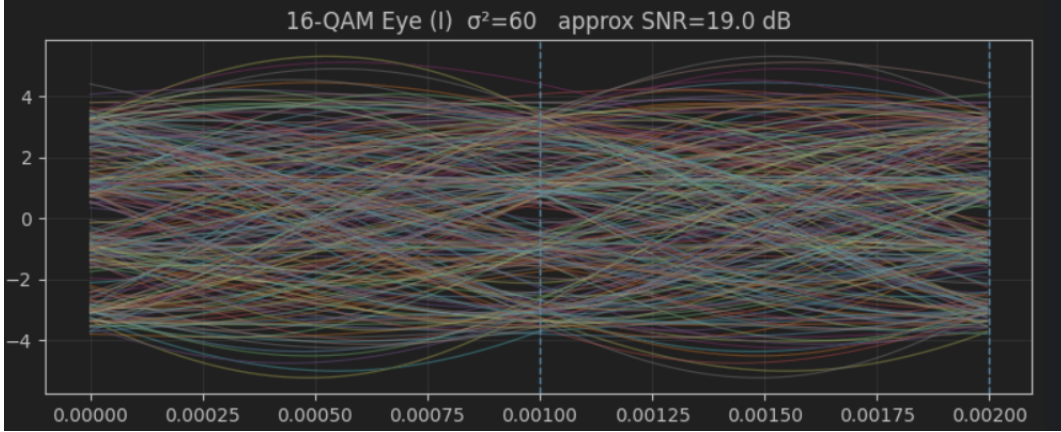


Figure 10: Eye diagram of 16-QAM at $\sigma^2 = 60$. Strong eye closure reflects high sensitivity to noise.

Overall interpretation. The eye diagrams clearly illustrate how increasing noise variance reduces both timing and amplitude margins. Among the three modulations, 4-FSK exhibits the most open eye and thus the highest robustness, QPSK shows moderate degradation, and 16-QAM suffers from severe eye closure. These time-domain observations are fully consistent with the BER and scatter-plot results presented in Parts (b) and (c), providing a unified physical interpretation of the system's performance under noise.

3.1 Implementation and Analysis

Implementation The project was implemented by coding each functional block of the digital communication system separately and validating its output before integration. For each modulation scheme (FSK, QPSK, and 16-QAM), dedicated functions were developed for modulation, demodulation, matched filtering, and symbol decision, while shared blocks such as pulse shaping, channel modeling, and noise generation were reused across all schemes.

Parts (a) through (d) were implemented incrementally. In each part, the output of the current block was verified numerically and graphically before proceeding to the next stage. After completing all blocks, they were connected to form a single end-to-end communication system. A fixed data sequence was transmitted through the system, and the received bits were compared with the transmitted bits to compute the bit error rate (BER). This procedure was repeated for different noise variances and channel lengths to ensure a fair and consistent comparison between modulation schemes.

3.1.1 BER Comparison Under Identical Noise Conditions Using the BER versus noise variance results obtained in Part (b) for each modulation scheme, the performance of FSK, QPSK, and 16-QAM was compared under identical noise conditions. At a noise variance of $\sigma^2 = 60$, the measured BER values are

$$FSK = 0.02375, \quad QPSK = 0.00975, \quad 16-QAM = 0.0168.$$

These results show that QPSK achieves the best BER performance at $\sigma^2 = 60$, followed by 16-QAM, while FSK exhibits the highest error rate. The superior performance of QPSK is due to its moderate constellation size, which provides a larger minimum Euclidean distance between symbols compared to 16-QAM, resulting in higher robustness against noise. Although 16-QAM is more spectrally efficient, its denser constellation increases sensitivity to noise, leading to a higher BER than QPSK at this noise level. FSK, while generally stable, demonstrates inferior BER performance under the same conditions.

(2) Eye Diagram Analysis at $\sigma^2 = 60$

To gain qualitative insight into timing sensitivity and noise margin, eye diagrams were examined at a fixed noise variance of $\sigma^2 = 60$. Figures ??–?? show representative eye diagrams for the considered modulation schemes.

- **FSK:** The eye representation derived from the matched-filter outputs exhibits a relatively clear opening around the sampling instant, indicating a strong noise margin and reliable symbol decision even at this noise level.
- **QPSK:** The eye diagrams for the in-phase and quadrature components (shown separately) remain reasonably open at the optimal sampling instant. This reflects balanced robustness and efficiency, with moderate sensitivity to noise.
- **16-QAM:** Due to the presence of multiple amplitude levels, the eye diagram is significantly more crowded. The reduced vertical eye opening indicates a smaller decision margin, explaining the higher BER observed for this modulation at $\sigma^2 = 60$.

These eye diagrams shown above in pervious visually confirm the BER trends observed in the quantitative analysis.

3.1.3 Effect of Channel Length on Modulation Performance The impact of channel length on modulation performance was evaluated by comparing BER values at multiple noise variances and transmission distances. The measured BER values for $\sigma^2 = [15, 30, 45, 60, 75, 90]$ indicate that the optimal modulation scheme varies with both noise level and channel length.

At $\sigma^2 = 60$, the BER results are

$$FSK = 0.0215, \quad QPSK = 0.03425, \quad 16-QAM = 0.0144,$$

showing that 16-QAM achieves the lowest BER at this noise level. The results further indicate that QPSK performs best at shorter transmission lengths (e.g., 0.5 km) but degrades significantly as the channel length increases, becoming the worst-performing scheme at longer distances (e.g., 2 km). In contrast, FSK exhibits relatively stable performance across different channel lengths, with BER changing slowly

as distance increases. For 16-QAM, the BER values corresponding to shorter and longer channel lengths become closer as noise variance increases, indicating reduced sensitivity to distance at higher noise levels.

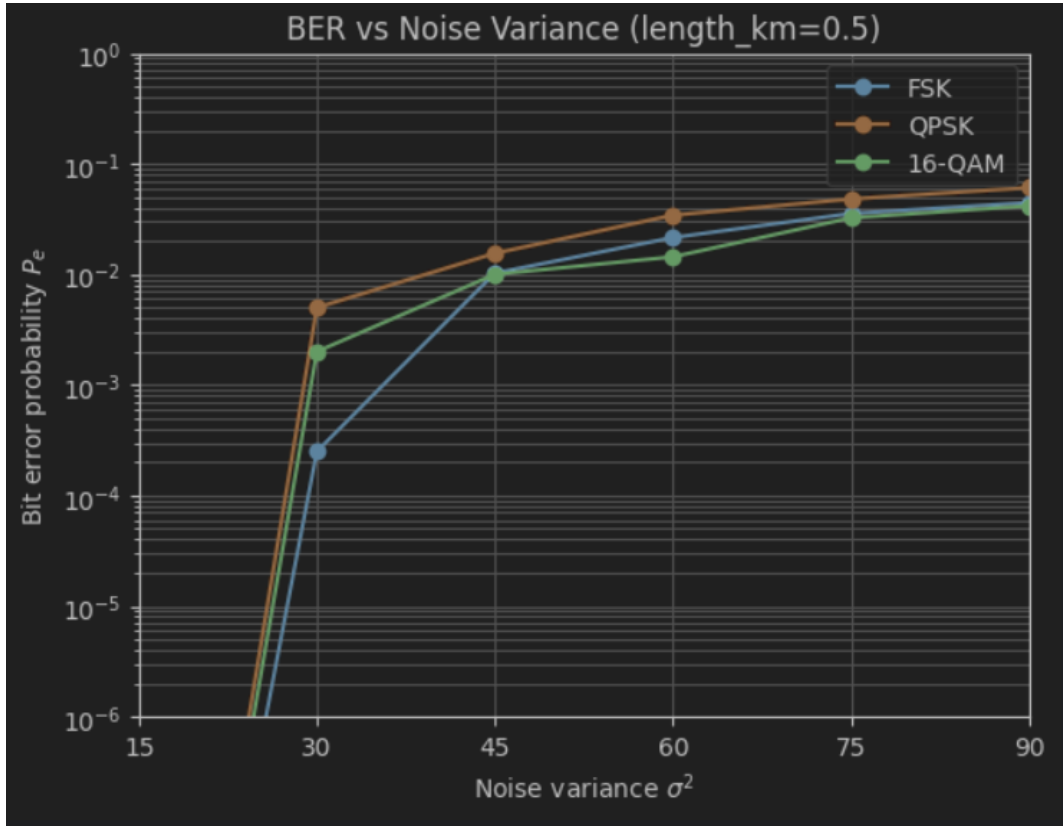


Figure 11: BER comparison of modulation schemes at 0.5 km

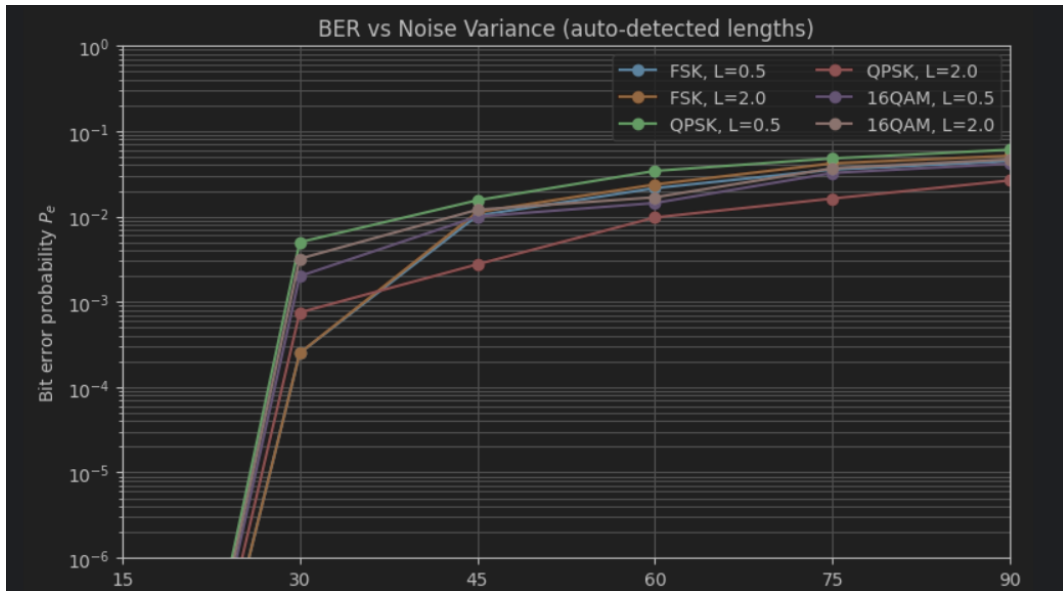


Figure 12: BER comparison of modulation schemes at 0.5 km and 2 km

4.1 sampling of the audio signal

in this part, the goal is to obtain a discrete-time speech sequence suitable for pcm processing. since the dominant speech bandwidth is approximately 300–3400 hz , the sampling rate must satisfy the nyquist condition:

$$f_s \geq 2f_{\max} \approx 2 \times 3400 = 6800 \text{ } hz.$$

therefore, we selected the standard telephony sampling rate $f_s = 8 \text{ } khz$.

implementation we recorded a minimum of 20 seconds of speech in python using an audio input stream, stored the samples as a one-dimensional array $x[n]$ with sampling rate f_s , and immediately re-played the same signal to verify correct acquisition. then, we produced: (i) a time-domain plot of $x[n]$ versus time $t = n/f_s$, and (ii) a frequency-domain analysis using the fft to confirm that most spectral energy lies in the speech band.

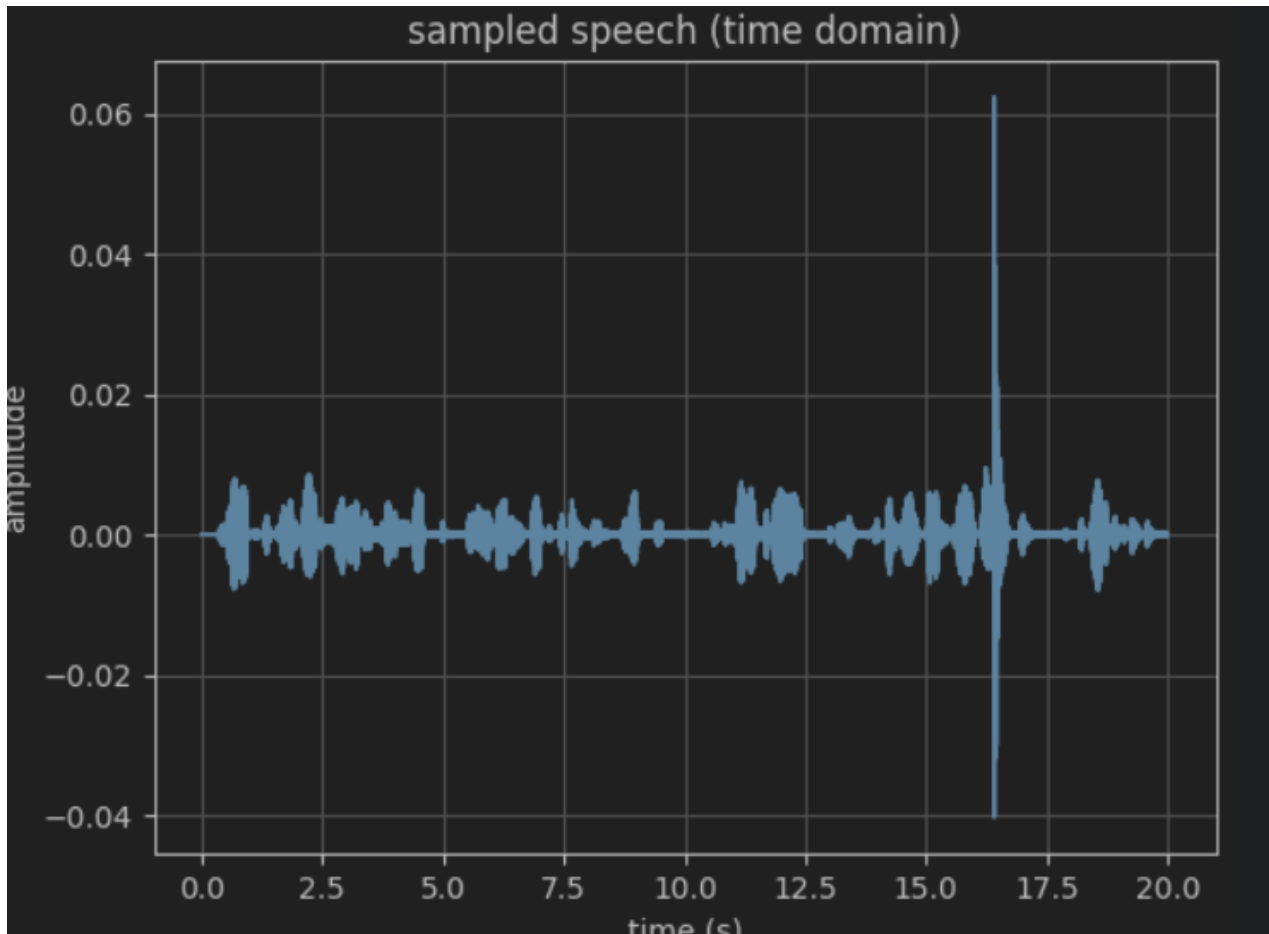


Figure 13: time-domain waveform of the recorded speech sampled at $f_s = 8 \text{ } khz$.

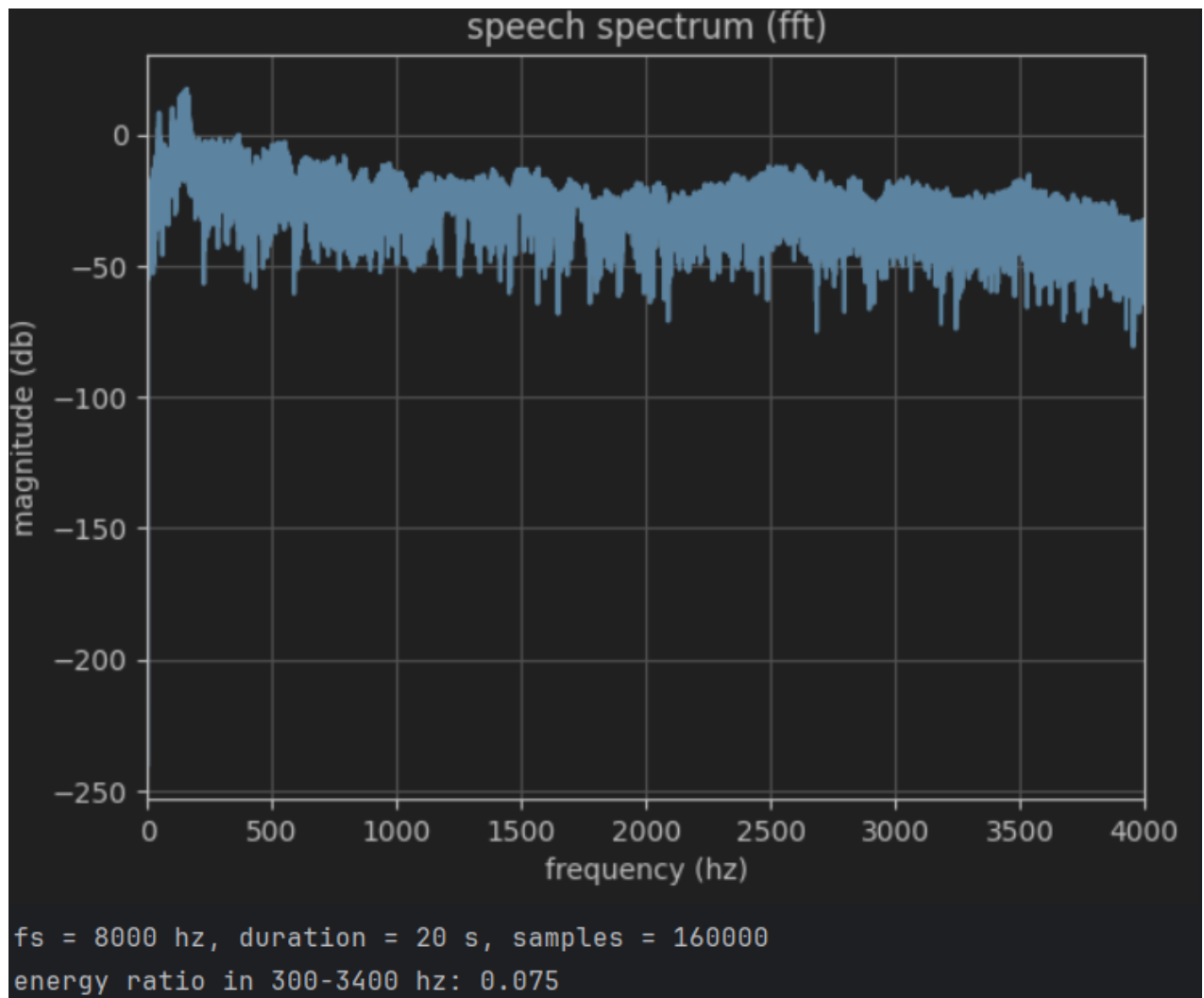


Figure 14: magnitude spectrum (fft) of the recorded speech, shown up to 4 kHz.

The spectrum is shown in logarithmic scale; due to the non-stationary nature of speech and the long observation window, the FFT exhibits broadband characteristics, while the dominant energy remains concentrated in the speech band below 4 kHz.

4.2 uniform quantization and pcm

after sampling in section 4.1, the discrete-time speech signal $x[n]$ must be mapped to a finite set of amplitude values in order to represent it digitally. in uniform quantization, the input dynamic range is divided into equally spaced levels. for an n_b -bit quantizer, the number of reconstruction levels is

$$L = 2^{n_b}.$$

using $n_b = 8$ bits (telephone pcm), the system bit rate becomes

$$R = f_s \times n_b = 8000 \times 8 = 64000 \text{ bps} = 64 \text{ kbps}.$$

uniform quantizer model let the observed signal range be $[x_{\min}, x_{\max}]$. the uniform step size is

$$\Delta = \frac{x_{\max} - x_{\min}}{L - 1}.$$

each sample is assigned an index

$$k[n] = \text{round}\left(\frac{x[n] - x_{\min}}{\Delta}\right), \quad k[n] \in \{0, 1, \dots, L - 1\},$$

and the reconstructed (quantized) sample is

$$x_q[n] = x_{\min} + k[n]\Delta.$$

the quantization error is defined as

$$e[n] = x[n] - x_q[n].$$

implementation in python we implemented a function `uniformquantizer` that takes the sampled signal $x[n]$ and the number of bits n_b , computes $(x_{\min}, x_{\max})$, forms Δ , generates the index sequence $k[n]$, and reconstructs $x_q[n]$. then, we formed the pcm bitstream by encoding each index $k[n]$ into its 8-bit binary representation. to evaluate quantization distortion, we plotted $x[n]$ and $x_q[n]$ on the same time window, plotted the error waveform, and plotted the error histogram.

quantization snr the quantization signal-to-noise ratio was computed as

$$\text{snr}_{q,\text{dB}} = 10 \log_{10} \left(\frac{E\{x^2[n]\}}{E\{e^2[n]\}} \right).$$

outputs : ” 4.2 results fs = 8000 hz nb = 8 bits/sample bit rate r = fs*nb = 64000 bps = 64.0 kbps quant range = [-0.0403, 0.0623] delta = 0.000402 snr (quantization) = 23.78 db pcm bitstream length = 1280000 bits”

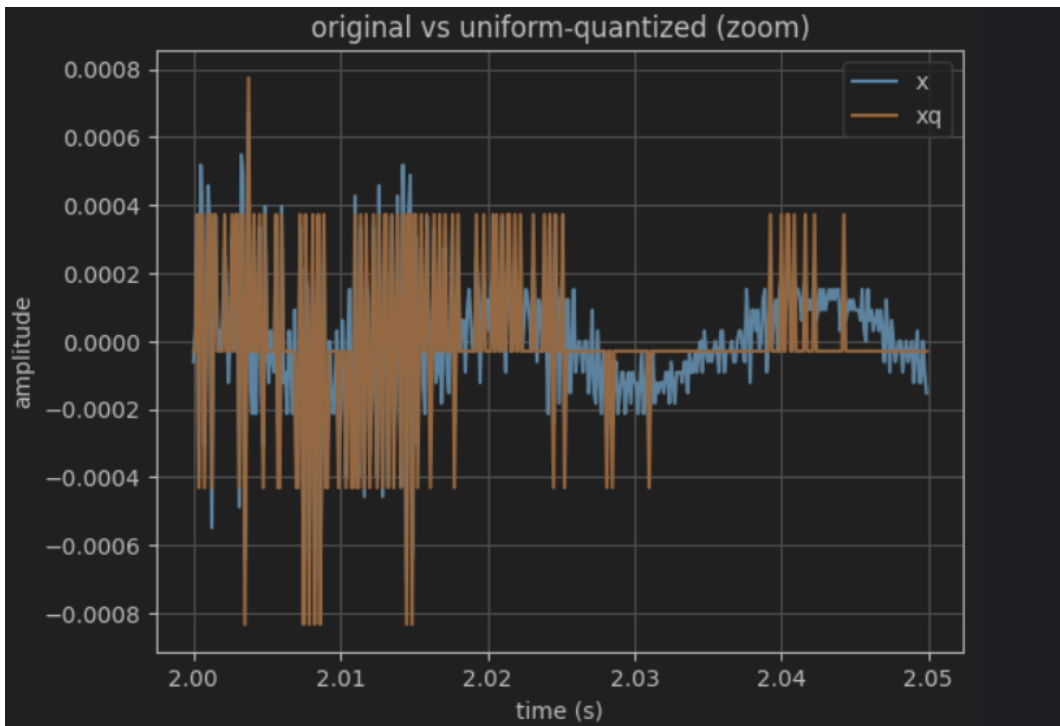


Figure 15: original sampled speech $x[n]$ and its uniformly quantized version $x_q[n]$ (zoomed time interval).

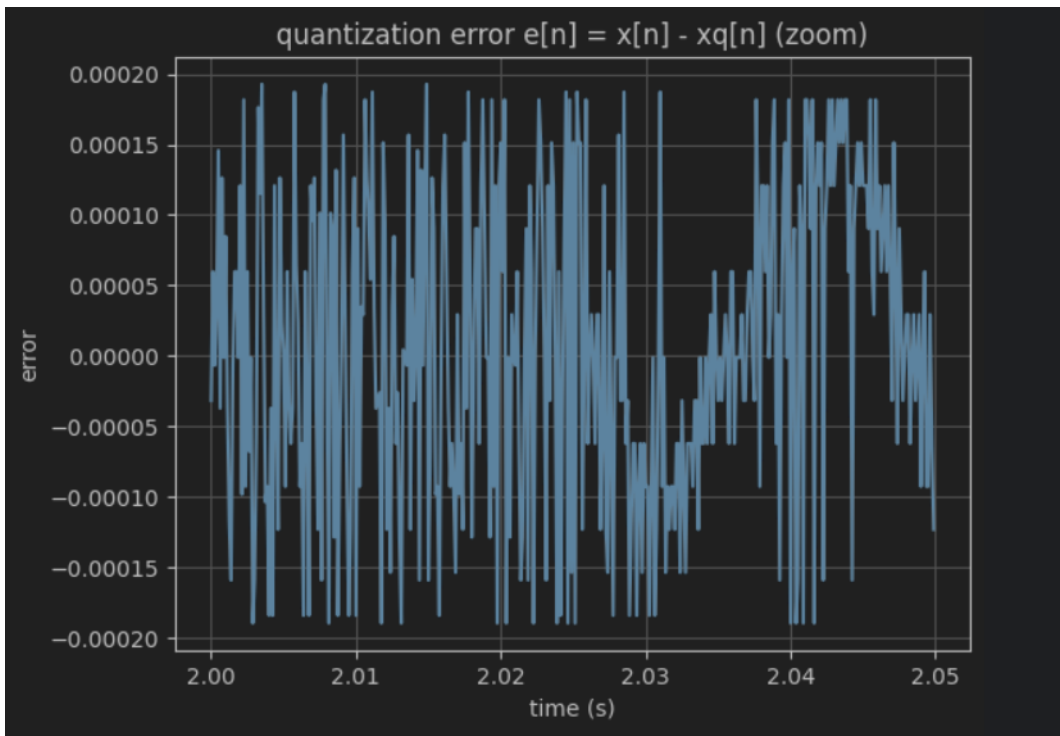


Figure 16: quantization error waveform $e[n] = x[n] - x_q[n]$ over the same zoomed interval.

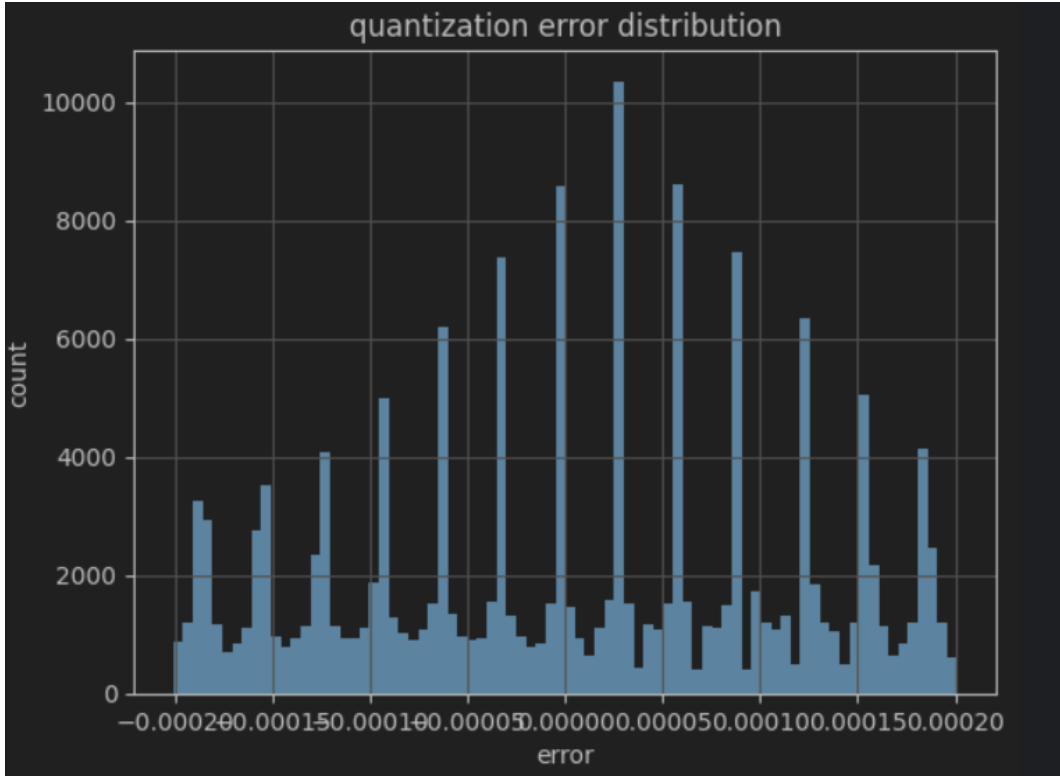


Figure 17: histogram of the quantization error, showing the distribution introduced by uniform quantization.

4.3 companding and non-uniform quantization

speech signals exhibit a highly non-uniform amplitude distribution, where a large fraction of samples have small magnitudes. uniform quantization therefore introduces relatively large distortion for low-amplitude speech components. to address this issue, companding is employed, which performs a non-linear amplitude mapping prior to quantization.

in this project, the μ -law companding technique was used. the μ -law compressor is defined as

$$y[n] = \text{sgn}(x_n[n]) \frac{\ln(1 + \mu |x_n[n]|)}{\ln(1 + \mu)},$$

where $x_n[n]$ is the normalized input signal and $\mu = 255$. after quantization, the inverse operation (expander) reconstructs the signal as

$$\hat{x}_n[n] = \text{sgn}(y[n]) \frac{1}{\mu} \left((1 + \mu)^{|y[n]|} - 1 \right).$$

implementation the sampled speech signal from section 4.1 was first normalized and passed through the μ -law compressor. the compressed signal was then uniformly quantized using 8 bits per sample. after quantization, the expander was applied and the signal was rescaled to its original amplitude range. the reconstructed signal was compared with the original waveform, and the quantization error was analyzed.

performance evaluation the quantization signal-to-noise ratio was computed as

$$\text{snr}_q = 10 \log_{10} \left(\frac{E\{x^2[n]\}}{E\{(x[n] - \hat{x}[n])^2\}} \right).$$

results show that μ -law companding reduces quantization distortion for low-amplitude speech samples compared to uniform quantization.

output :” 4.3 mu-law results fs = 8000 hz bits = 8 mu = 255.0 snr (mu-law) = 37.41 db snr (uniform) = 23.78 db ”

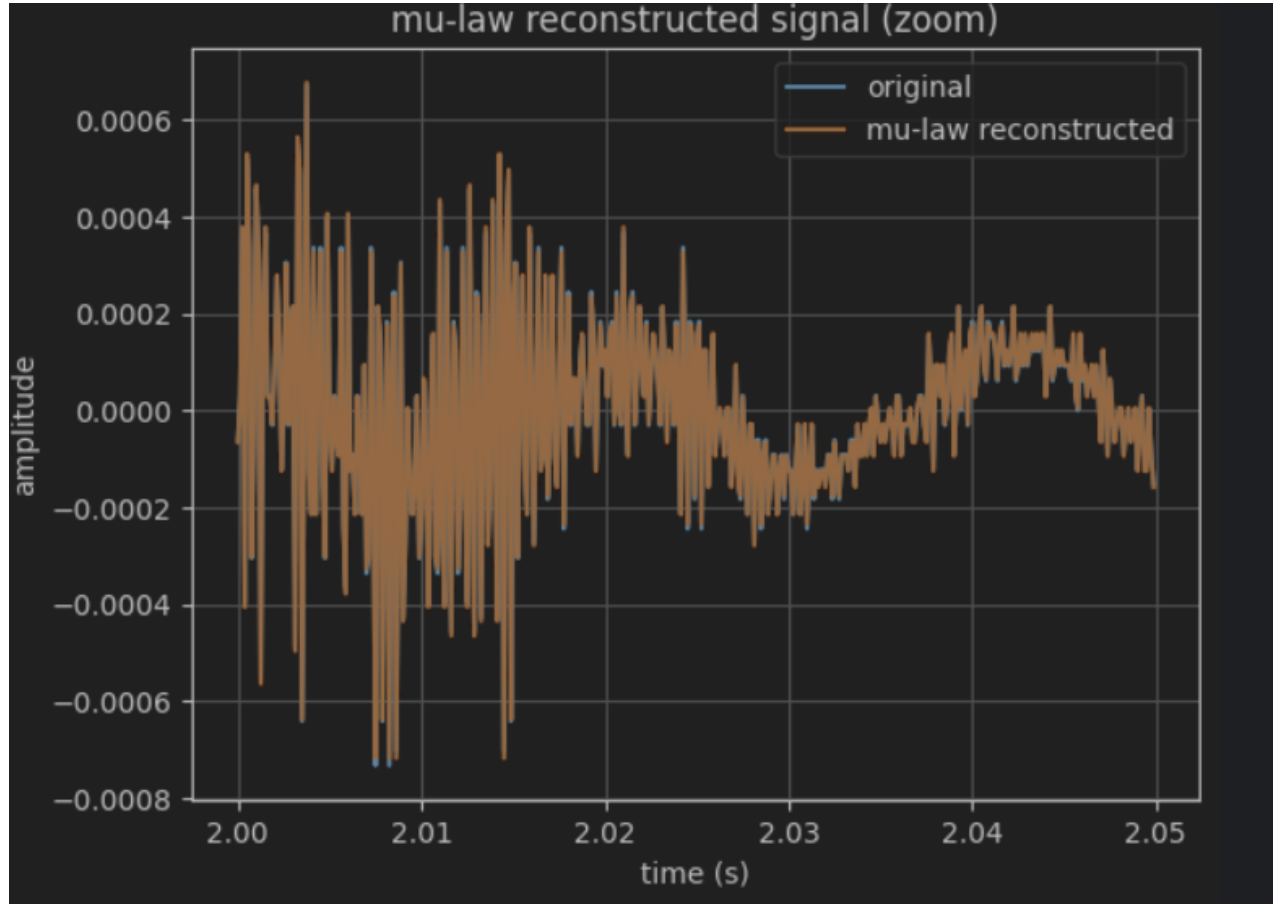


Figure 18: original and μ -law reconstructed speech signals over a short time interval.

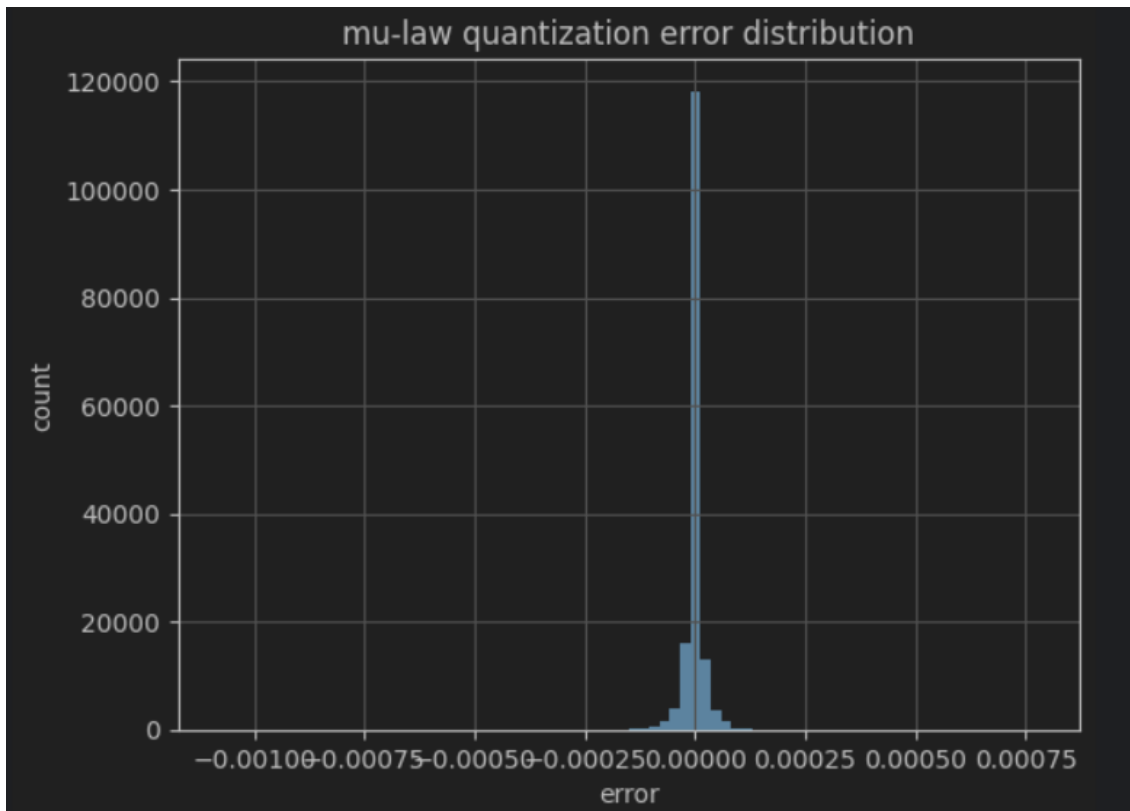


Figure 19: quantization error distribution obtained using μ -law companding.

4.4 comparison and analysis of results

(1) quantization error distribution the quantization error distributions for uniform quantization and μ -law companding were compared using error histograms. uniform quantization produces an error distribution that is approximately uniform within $\pm\Delta/2$, independent of signal amplitude. in contrast, the μ -law error distribution exhibits a higher concentration of small errors, indicating improved resolution for low-amplitude speech samples.

(2) effect on low-amplitude signals speech signals contain a large number of low-energy samples, particularly during unvoiced sounds and silence intervals. uniform quantization assigns equal resolution across the entire amplitude range, which leads to relatively large distortion for these small-amplitude components. μ -law companding applies a logarithmic compression prior to quantization, effectively allocating more quantization levels to low-amplitude regions. as a result, weak speech components are preserved more accurately and quantization noise is perceptually reduced.

(3) listening quality comparison a subjective listening test was performed by replaying the reconstructed speech signals obtained from uniform quantization and μ -law companding. the uniformly quantized signal exhibits audible distortion, particularly during low-energy speech segments. in contrast, the μ -law reconstructed signal closely follows the original waveform and provides noticeably improved

perceptual quality. this observation is consistent with the measured signal-to-noise ratios:

$$\text{snr}_{\text{uniform}} = 23.78 \text{ db}, \quad \text{snr}_{\mu\text{-law}} = 37.41 \text{ db}.$$

the higher snr achieved by μ -law companding directly translates into better subjective audio quality. , the construction with uniform steps could be understood sometimes , but had a lot of noise and my sound would go off for a like a peroid of 2-3 seconds time to time , but miu law result was crystall clear! all voices are in the uploaded file!

(4) practical relevance the results demonstrate that companding significantly improves speech quality without increasing the bit rate. by reducing quantization distortion for low-amplitude signals, μ -law companding achieves superior performance compared to uniform quantization. this explains its widespread use in practical pcm-based telecommunication systems, where efficient bandwidth usage and acceptable speech quality are critical.

Figure 20: comparison of reconstructed waveforms showing that the μ -law signal tracks the original speech more accurately than uniform quantization.

5.2)

in this part, we computed the discrete auto-correlation and cross-correlation of complex zadoff–chu (zc) sequences. the zc sequence generator from section 5.1 was used to create multiple length- n sequences with chosen root indices u . then, for each case, we evaluated the correlation functions and visualized their magnitudes versus lag.

implemented definitions. for two discrete-time signals $x[n]$ and $y[n]$, we implemented the full (non-circular) cross-correlation as

$$r_{xy}[k] = \sum_n x[n] y^*[n - k], \quad (3)$$

and the auto-correlation as $r_{xx}[k] = r_{xy}[k] \big|_{y=x}$. in python, this was realized using a single reusable function: `cross_correlation(x,y)` (full mode), and `auto_correlation(x)`.

5.2.3)

In this section, the correlation properties of Zadoff–Chu (ZC) sequences are examined through numerical simulation. For each considered case, both the auto-correlation and cross-correlation magnitude responses are evaluated and illustrated. All correlations are computed using linear (non-circular) correlation on finite-length sequences.

Auto-correlation Results

Figures 21 and ?? show the magnitude of the auto-correlation $|R_{xx}[k]|$ for two ZC sequences with identical length but different root indices.

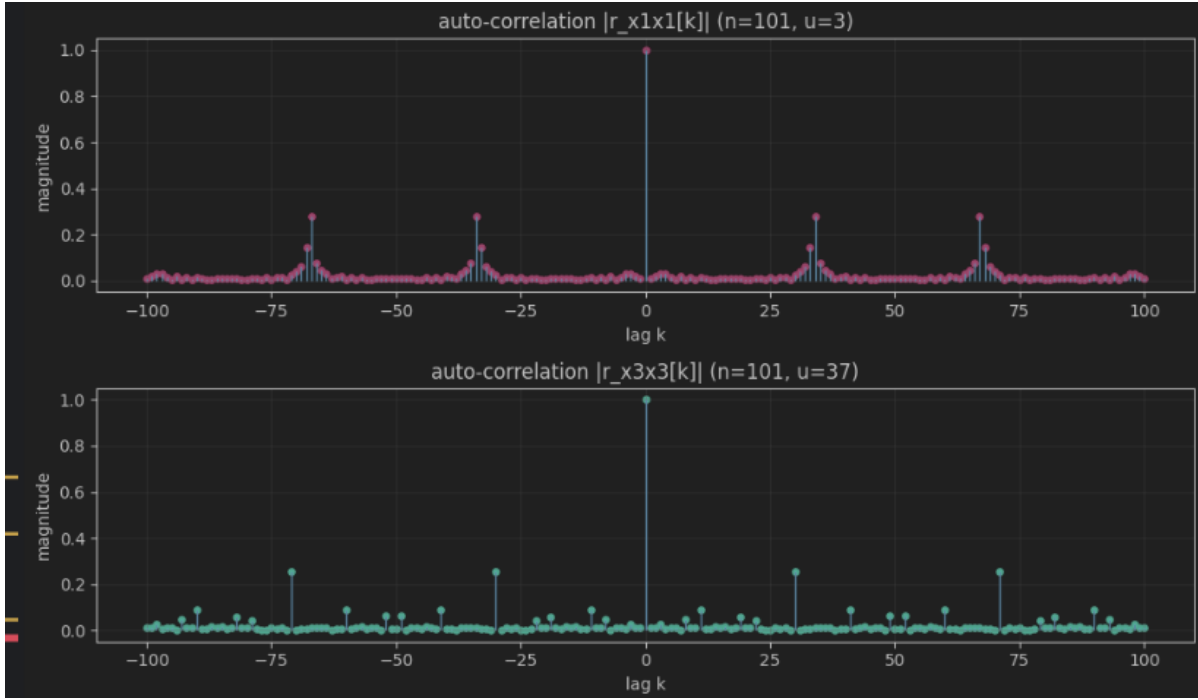


Figure 21: Auto-correlation magnitude of a Zadoff–Chu sequence (example 1).

In both cases, a dominant peak is clearly observed at zero lag ($k = 0$), while the correlation values at other lags remain significantly smaller. This behavior indicates that each sequence exhibits a strong similarity only with a time-aligned version of itself. The small sidelobes visible at nonzero lags are a consequence of linear correlation and finite sequence length, as partial overlap occurs when the sequence is shifted.

The presence of a single, well-defined peak demonstrates that the ZC sequence provides an accurate and unambiguous timing reference, which is a critical requirement for synchronization and detection tasks at the receiver.

Cross-correlation Results

Figures 22 and ?? illustrate the magnitude of the cross-correlation $|R_{xy}[k]|$ between pairs of ZC sequences.

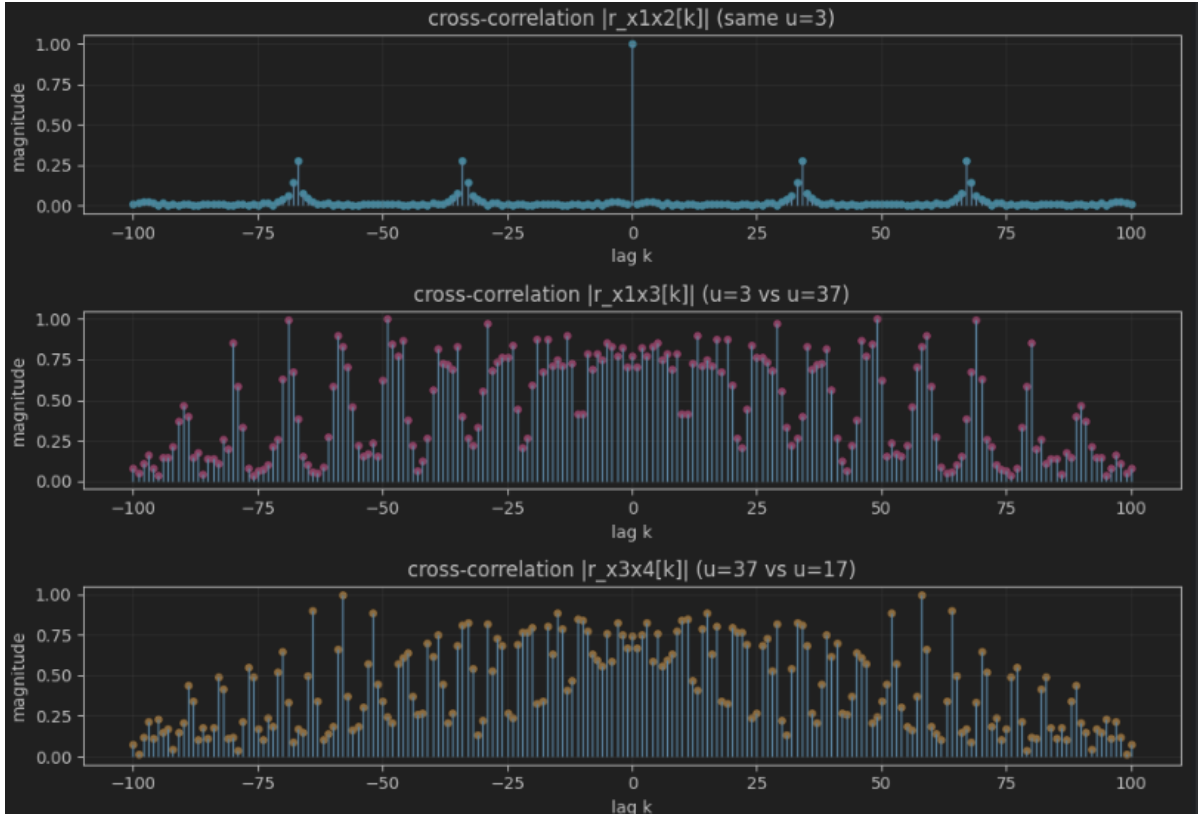


Figure 22: Cross-correlation magnitude between two identical ZC sequences (same root index).

When the two sequences are identical, the cross-correlation response resembles the auto-correlation result, with a dominant peak at zero lag. In contrast, when different root indices are used, the cross-correlation exhibits no pronounced peak at any lag. Instead, the correlation energy is distributed across multiple lags with relatively low magnitude.

This distinction highlights a fundamental property of ZC sequences: sequences generated with different roots remain easily distinguishable at the receiver, since their cross-correlation does not produce false timing peaks. The smooth envelope observed in the cross-correlation plots is a direct result of linear correlation and finite sequence length, rather than a degradation of orthogonality.

5.2.4)

Based on the presented figures, two key properties of Zadoff–Chu sequences can be identified.

First, the strong impulse-like auto-correlation ensures reliable timing detection. A receiver can accurately determine the arrival time of a transmitted sequence by locating the dominant correlation peak, even in the presence of noise.

Second, the absence of significant peaks in the cross-correlation between sequences with different root indices prevents ambiguity between multiple users. In random access scenarios, several user equipments may transmit different preamble sequences simultaneously. The low cross-correlation guarantees that the detection of one user’s preamble is not corrupted by others.

These two properties make Zadoff–Chu sequences particularly suitable for the Random Access Channel

(RACH) procedure in LTE and 5G systems. They enable robust synchronization, reliable user identification, and collision-resistant access, which are essential requirements for initial network access in cellular communication systems.

5.3.1 requirement (1): embed mib into a 1000-bit stream

To simulate the presence of a broadcast system pattern (MIB) inside an unknown downlink bitstream, we first generate a random binary sequence of length $N = 1000$ using a pseudo-random generator with a fixed seed (for reproducibility). Then, we define a fixed MIB pattern of length $L_{mib} = 24$ bits and overwrite a contiguous block of the random sequence starting at an arbitrary index k_0 .

The code creates:

- `bits`: a random vector in $\{0, 1\}^{1000}$
- `mib`: a fixed 24-bit pattern
- `mib_pos` (k_0): the insertion index, which is the main parameter that controls the MIB position

A range check enforces $0 \leq k_0 \leq N - L_{mib}$ to ensure the 24-bit block fits inside the 1000-bit stream.

what this enables next. After embedding the MIB, the resulting stream is ready to be modulated (QPSK), transmitted through the same channel/noise model as previous sections, and finally detected by correlation at the receiver to estimate $\hat{k}_0$.

” total bits: 1000 mib length: 24 mib position: 455 mib bits: 101101001101001010011100 segment check: 101101001101001010011100”

5.3.1: generate a 1000-bit stream and insert the 24-bit mib

A random binary sequence of length $N = 1000$ is generated and a fixed 24-bit pattern (representing the MIB) is embedded at an arbitrary position k_0 . In the implementation, the position is controlled by the parameter `mib_pos`, and it must satisfy

$$0 \leq k_0 \leq N - L_{mib}, \quad L_{mib} = 24.$$

After insertion, we plot the full 1000-bit stream and mark the MIB start/end indices to visually verify correct placement.

Figure 23: 5.3.1: the 1000-bit stream with the embedded 24-bit MIB block (dashed lines show the start and end of the MIB).

5.3.2: qpsk modulation and transmission through the channel

The bitstream (with the embedded MIB) is transmitted using QPSK. Bits are grouped into pairs (b_0, b_1) and mapped to symbols

$$00 \rightarrow (+1, +1), \quad 01 \rightarrow (-1, +1), \quad 11 \rightarrow (-1, -1), \quad 10 \rightarrow (+1, -1),$$

so that $s_k = I_k + jQ_k$. A rectangular pulse of duration T_b is used for pulse shaping, and the complex baseband waveform is upconverted to passband via

$$x_c(t) = I(t) \cos(2\pi f_c t) - Q(t) \sin(2\pi f_c t).$$

The passband signal is then passed through the same channel and AWGN blocks used in previous parts (we do not re-implement them). At the receiver, coherent downconversion and low-pass filtering produce the recovered baseband components $y_I(t)$ and $y_Q(t)$.

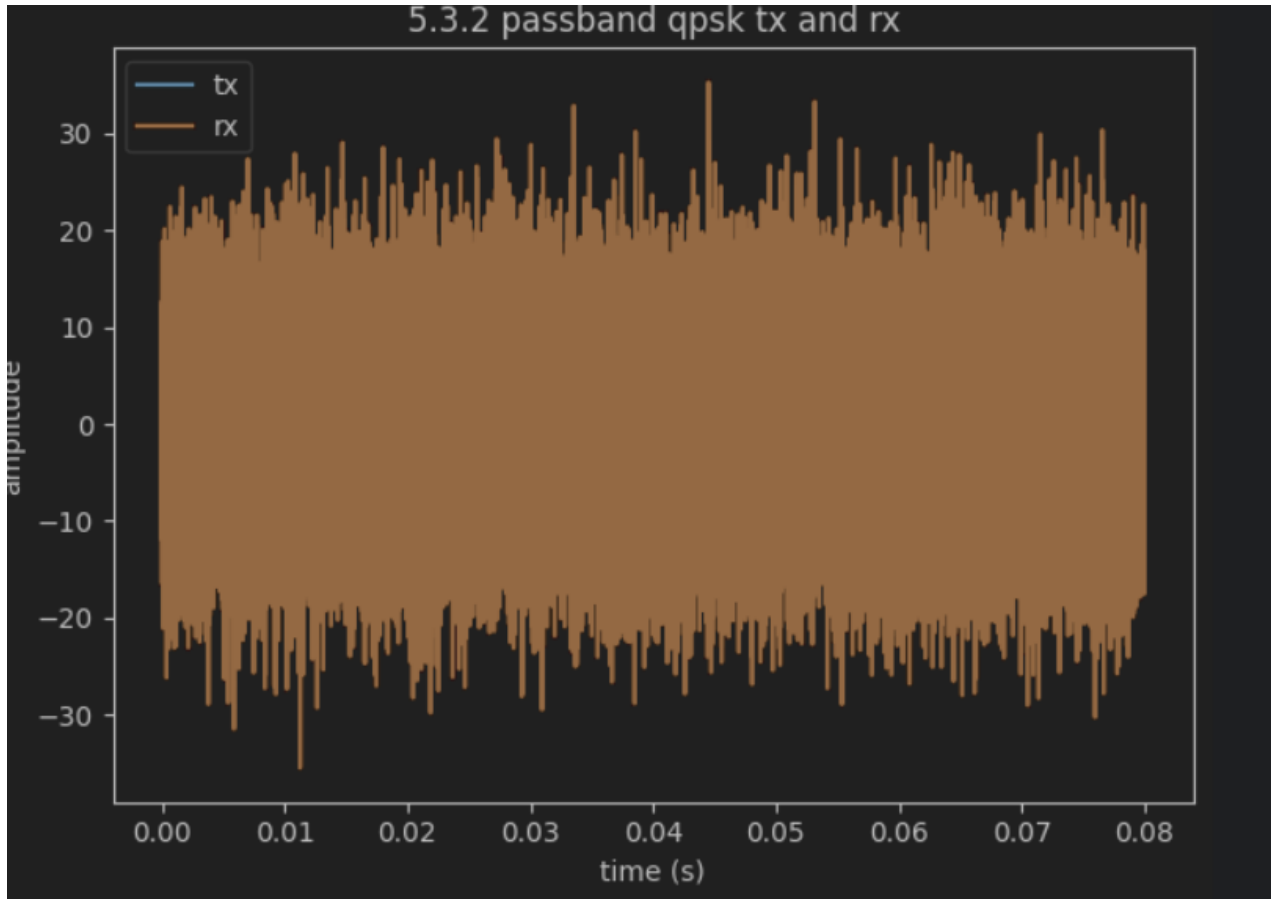


Figure 24: 5.3.2: transmitted passband QPSK signal $x_c(t)$ and received signal $r(t)$ after channel + AWGN.

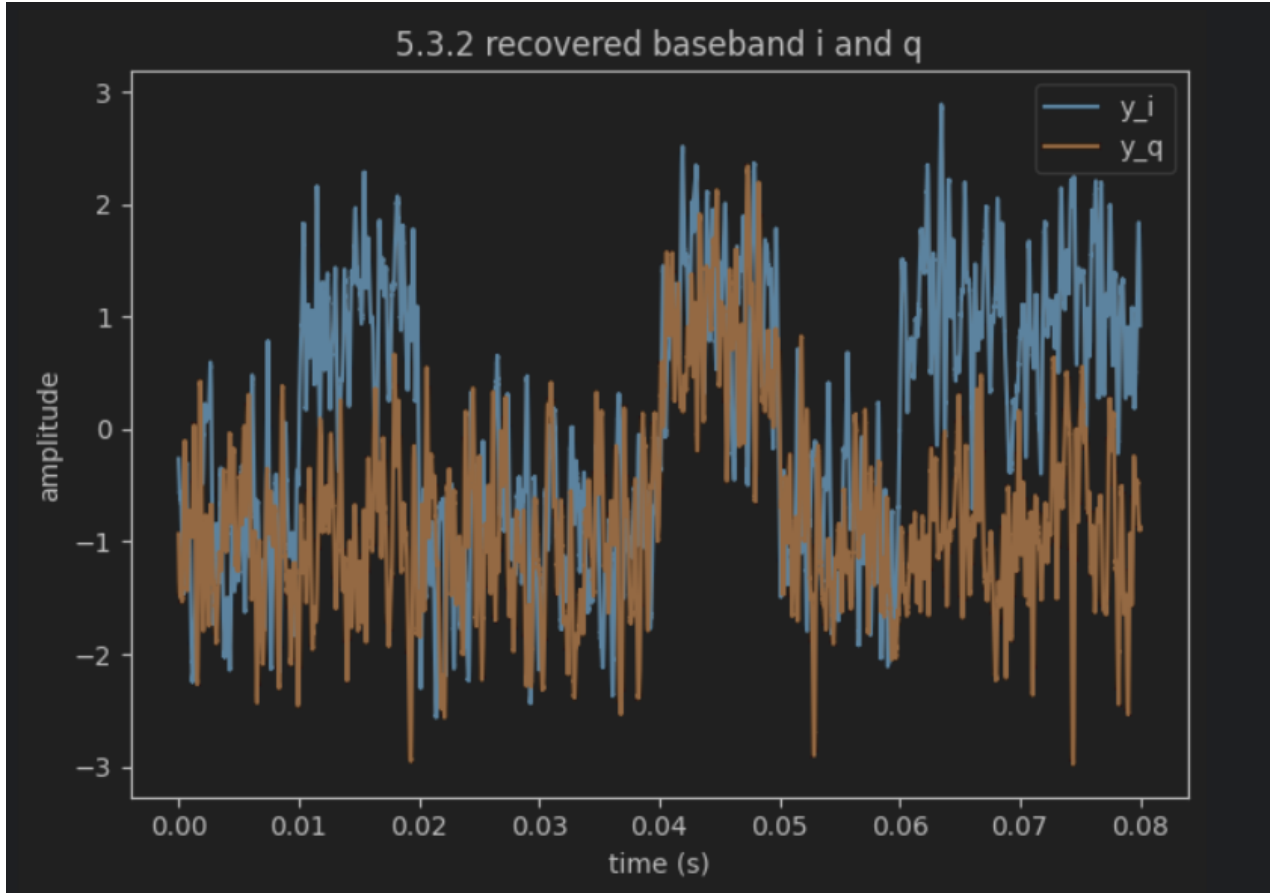


Figure 25: 5.3.2: recovered baseband components $y_I(t)$ and $y_Q(t)$ after coherent demodulation and low-pass filtering.

5.3.3: mib position detection using correlation

After QPSK transmission and coherent demodulation (Requirement 5.3.2), the receiver has access to the recovered baseband components $y_I(t)$ and $y_Q(t)$. We form the complex baseband sequence

$$y_{rx}[n] = y_I[n] + jy_Q[n],$$

and detect the embedded MIB by correlating $y_{rx}[n]$ with a locally generated reference waveform corresponding to the known 24-bit MIB pattern.

reference generation and alignment ambiguity. Because QPSK groups bits in pairs, the MIB may begin at either an even-bit boundary or an odd-bit boundary relative to the symbol pairing. Therefore, two references are built:

- **align 0:** modulate the MIB bits directly using the same QPSK transmitter and pulse shaping used in the link.
- **align 1:** prepend one dummy bit (one-bit shift) and then modulate, representing an odd-bit start.

correlation-based detection. Using the cross-correlation function implemented in Section 5.2, we compute

$$R[k] = \sum_n y_{rx}[n] y_{mib}^*[n-k].$$

The estimated MIB start location corresponds to the lag k that maximizes $|R[k]|$. The chosen alignment is the one that produces the larger peak magnitude. The peak index is then mapped back to the corresponding start position in the bitstream.

5.3.3–5.3.7 MIB Detection and Random Access (RACH) Procedure

In this section, a simplified initial-access scenario is simulated. A length-1000 random bitstream is assumed to be transmitted over the same channel and AWGN conditions used in previous parts, using QPSK modulation. A fixed 24-bit pattern representing the *MIB* is embedded inside the bitstream, and the receiver (UE) must (i) detect the MIB location via correlation, then (ii) perform a simplified Random Access procedure by selecting and transmitting one of 64 Zadoff–Chu (ZC) preambles, (iii) detecting the received ZC preamble, (iv) encoding the detected preamble index into a 6-bit number and modulating it, and finally (v) declaring successful attachment if the detected preamble matches the transmitted one.

5.3.3 Detecting the MIB location using correlation

Let $\mathbf{b} \in \{0, 1\}^{1000}$ denote the transmitted bitstream and let $\mathbf{m} \in \{0, 1\}^{24}$ denote the known MIB pattern inserted at an (unknown-to-receiver) position k_0 . After QPSK transmission, channel attenuation, and AWGN addition, the receiver obtains a noisy passband waveform $r(t)$. Coherent demodulation produces baseband I/Q waveforms $y_I(t)$ and $y_Q(t)$.

To apply correlation in the discrete domain, symbol-rate sampling is performed (with sampling period T_b) to obtain decision samples $\{\hat{I}_k, \hat{Q}_k\}$, followed by hard decisions to obtain an estimated bit sequence $\hat{\mathbf{b}}$. With the specific Gray-like mapping used in our QPSK implementation, the bit decisions are extracted by sign-thresholding:

$$\hat{b}_{0,k} = 1\{\hat{Q}_k < 0\}, \quad \hat{b}_{1,k} = 1\{\hat{I}_k < 0\},$$

and the recovered bitstream is formed by interleaving $\hat{b}_{0,k}$ and $\hat{b}_{1,k}$.

Since correlation is most effective when the pattern has zero-mean, the binary sequences are mapped to bipolar form

$$\tilde{b}[n] = 2\hat{b}[n] - 1 \in \{-1, +1\}, \quad \tilde{m}[n] = 2m[n] - 1 \in \{-1, +1\}.$$

The MIB position is then estimated by the peak of the sliding (valid) correlation:

$$C[k] = \sum_{n=0}^{23} \tilde{b}[k+n] \tilde{m}[n], \quad \hat{k}_0 = \arg \max_k C[k].$$

A strong peak at $\hat{k}_0$ indicates that the received bits contain a segment highly consistent with the known MIB pattern, while the remaining offsets produce smaller correlation due to random mismatch. In other words, correlation converts “pattern search” into a robust peak-detection problem.

5.3.4 Random selection and transmission of a Zadoff–Chu preamble

After successful MIB detection, the UE randomly selects one preamble from a set of 64 available ZC sequences. A ZC sequence of length N and root index u is defined by

$$x_u[n] = \begin{cases} \exp(-j\pi un(n+1)/N), & \text{Nodd} \\ \exp(-j\pi un^2/N), & \text{Neven} \end{cases}, \quad n = 0, 1, \dots, N-1,$$

where u is chosen coprime with N . The key property exploited in random access is that ZC sequences exhibit an impulse-like autocorrelation and low cross-correlation for different roots, which enables reliable preamble detection by correlation even under noise.

In the implementation, a prime length (e.g., $N = 101$) is chosen so that at least 64 valid roots exist. The UE uniformly selects an index $i_{tx} \in \{0, \dots, 63\}$ corresponding to one root u_{tx} , then transmits $x_{u_{tx}}[n]$ through the same attenuation model and AWGN.

5.3.5 Receiver-side ZC identification and 6-bit index formation

At the receiver, the goal is to identify which of the 64 candidate ZC sequences was transmitted. Denote the received preamble by $z_{rx}[n]$. A correlation metric is computed against each candidate $\{x_{u_i}[n]\}_{i=0}^{63}$:

$$\rho_i = \max_k \left| \sum_n z_{rx}[n] x_{u_i}^*[n-k] \right|, \quad \hat{i} = \arg \max_i \rho_i.$$

The detected index $\hat{i}$ is then represented as a 6-bit integer (since $2^6 = 64$). This produces a control message bit-vector

$$\mathbf{d} = \text{bin}_6(\hat{i}) \in \{0, 1\}^6,$$

which is the information that would be conveyed to the next signaling stage in a simplified access procedure.

Practically, detection errors can occur when noise is strong or when the correlation peaks of multiple candidates become close. Therefore, $\hat{i}$ need not always equal the transmitted i_{tx} ; this is expected behavior in a noisy random-access channel.

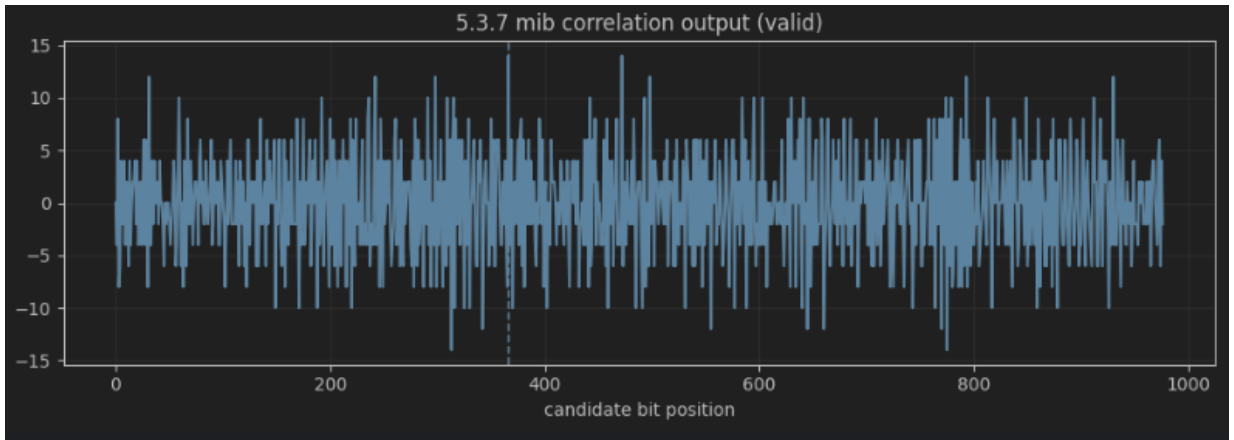


Figure 26: Example ZC detection metric ρ_i across the 64 candidates. The maximum indicates the detected preamble index $\hat{i}$.

5.3.6 Attachment decision (“Attach Completed”)

The simplified success criterion is:

$$\text{AttachCompleted} \iff \hat{i} = i_{tx}.$$

If the detected ZC index equals the transmitted one, the receiver declares a successful random access attempt; otherwise, `Attach Failed` is printed. In a realistic LTE/5G system, additional contention resolution and higher-layer checks would follow, but this simplified rule captures the fundamental requirement: *correct preamble identification*.

In our simulation, observing `Attach Failed` is not inherently an error; it indicates that under the chosen noise variance the ZC detector made an incorrect decision. Reducing noise power or increasing preamble length would increase the detection probability, which is consistent with correlation-based detection theory.

5.3.7 Required plots and interpretation

Finally, the following signals are plotted to visualize the full procedure:

- **MIB modulated signal:** QPSK passband waveform corresponding to the 24-bit MIB pattern.
- **Transmitted ZC preamble:** baseband complex ZC sequence selected by the UE.
- **Correlation output for MIB detection:** the sliding correlation $C[k]$ whose peak estimates the MIB location.
- **6-bit modulated signal:** QPSK passband waveform corresponding to the 6-bit detected preamble index.

Because the passband QPSK signal is sampled at a high rate (e.g., $f_s = 1$ MHz) over multiple milliseconds, the waveform can visually appear as a dense “filled” region when plotted over a long time window. For clarity, the reported figures use a short time zoom (e.g., a few milliseconds) or decimated samples, so that carrier oscillations and symbol transitions are visible.

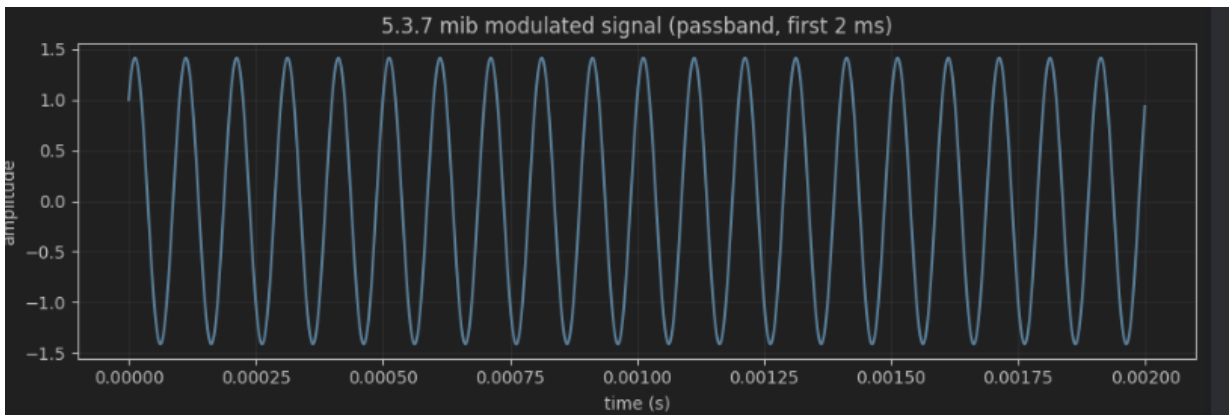


Figure 27: Passband QPSK waveform of the MIB pattern (zoomed time window for visibility).

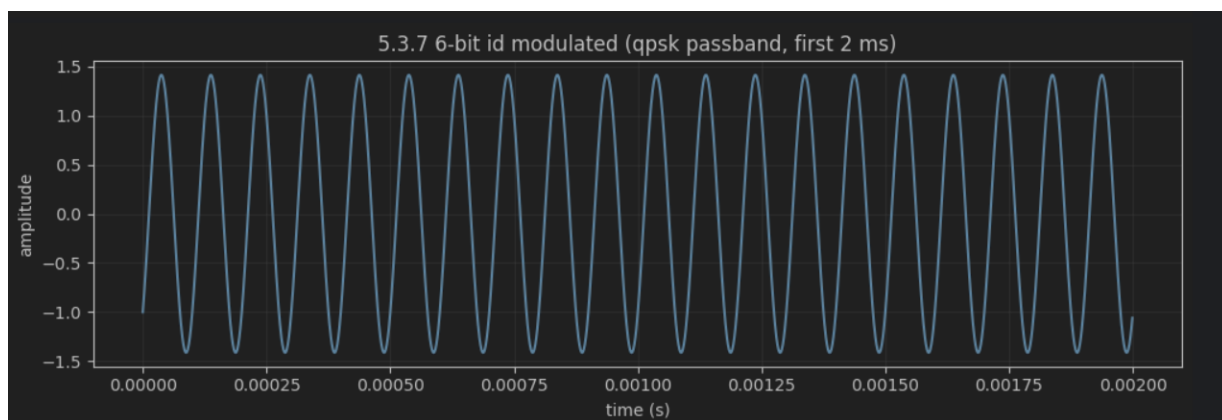
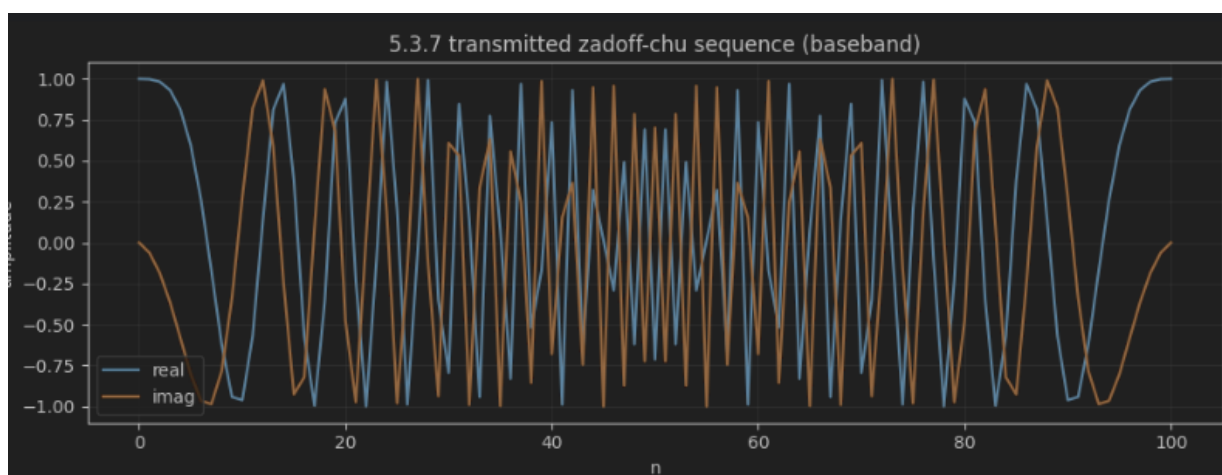


Figure 28: Passband QPSK waveform of the 6-bit modulated index message (zoomed).



Overall RACH signal flow. The simulated random access procedure can be summarized by the following sequence of processing steps:

1. Generation of a length-1000 binary sequence with an embedded 24-bit MIB pattern.
2. QPSK modulation of the bitstream and transmission through the attenuating channel with AWGN.
3. Coherent demodulation and recovery of the received bit sequence at the UE.
4. Correlation-based detection of the MIB location in the recovered bitstream.
5. Random selection and transmission of a Zadoff–Chu preamble from a set of 64 candidates.
6. Correlation-based identification of the received Zadoff–Chu preamble at the receiver.
7. Binary encoding of the detected preamble index as a 6-bit sequence and QPSK modulation of this control message.

Overall, this experiment demonstrates two cornerstone ideas in cellular initial access: (i) *known-pattern detection by correlation* (used here for MIB localization), and (ii) *preamble-based contention access*

using correlation-friendly sequences (modeled here using a 64-option Zadoff–Chu bank). The observed outcomes (clear correlation peak for the MIB, and occasional ZC index errors under higher noise) follow directly from matched-correlation detection theory.